答案册

《精选题集》全部答案与解析（408 + 802真题；题集答案暂缺）

408真题 151 题　802真题2024 23 题　802真题2025 23 题　802真题2026 26 题

合计 **223** 题

编排：按题型分区，题型内按章排列

编制说明

- 编排：全书按题型分区，同一题型内按章节先后排列；目录与 PDF 书签即按此两级组织。
- 本册收录《精选题集》范围内有答案的全部题目（408 + 802真题）；题集（含一档）的官方答案区暂未识别入库，故无答案，待补后另出修订。
- 题号 #N 与《精选题集》一一对应；保留题目 ID 以便检索对照。
- 红框【勘误·以修正版为准】：答案 PDF 存在错误或存疑、经核对后修正的题目（详见题库 README 第六节），讲题/刷题一律以修正版为准。
- 802 真题为回忆版，个别题图缺失、题面不全，备注中如实标注。

伴梦研途 · 重邮802数据结构代码课

目录

选择题 128 题

第一章　绪论 4 题　　第二章　线性表 8 题　　第三章　栈队列 10 题　　第四章　串 3 题

第五章　数组广义表 6 题　　第六章　树 27 题　　第七章　图 28 题　　第八章　查找 18 题

第九章　排序 24 题

填空题 29 题

第二章　线性表 1 题　　第三章　栈队列 7 题　　第四章　串 2 题　　第五章　数组广义表 5 题

第六章　树 8 题　　第七章　图 1 题　　第八章　查找 2 题　　第九章　排序 3 题

程序阅读题 14 题

第一章　绪论 3 题　　第二章　线性表 5 题　　第三章　栈队列 2 题　　第六章　树 2 题

第七章　图 2 题

综合应用题 43 题

第二章　线性表 3 题　　第三章　栈队列 3 题　　第五章　数组广义表 2 题　　第六章　树 8 题

第七章　图 13 题　　第八章　查找 11 题　　第九章　排序 3 题

算法设计题 9 题

第二章　线性表 2 题　　第六章　树 4 题　　第七章　图 3 题

选择题 128 题

第一章　绪论 4 题

#9 408真题

408-2017-1

答案

B

解析

`sum += ++i;` 相当于 `++i; sum = sum + i;` 进行到第 k 趟循环，`sum = (1+k)*k/2`。显然需要进行O(n1/2)趟循环，因此这也是该函数的时间复杂度。

考点　复杂度分析

#10 408真题

408-2019-1

答案

B

解析

假设第 k 次循环终止，则第 k 次执行时，(x+1)2>n，x 的初始值为 0，第 k 次判断时，x=k-1，即k2>n，k>n1/2,，因此该程序段的时间复杂度为O(n1/2)。

考点　复杂度分析

#11 408真题

408-2022-1

答案

B

解析

当外层循环的变量i取不同值时，内层循环就执行多少次，因此总循环次数为的所有取值之和。假设外层循环共执行k次，当i=1,2,4,8,…,2k−1(2k−1<n≤2k)时，内层循环执行i次，因此总循环次数T=1+2+4+8+…+2k−1=2k−1即n<T<2n，时间复杂度为O(n)。

考点　复杂度分析

#12 408真题

408-2025-1

答案

B

解析

外层循环的条件是i2<n，因此 i 的最大值为n。内层循环的的次数同样与 i 相同。所以总的循环次数为nn=n，时间复杂度为O(n)。

考点　复杂度分析

第二章　线性表 8 题

#15 408真题

408-2016-1

答案

D

解析

根据存储状态，单链表的结构如下图所示。

〔结构示意未收录〕其中“链接地址”是指结点 next 所指的内存地址。当结点 f 插入后，a 指向 f, f 指向 e, e 指向 b。显然 a、e 和 f 的“链接地址”分别是 f、b 和 e 的内存地址，即 1014H、1004H 和 1010H。

考点　链表

#16 408真题

408-2016-2

答案

D

解析

参考单链表删除操作，这种题为送分题。

考点　链表

#17 408真题

408-2021-1

答案

D

解析

删除头结点后一个结点的基本流程为 `q = h->next; h->next = q->next; free(q);` ，即通过指针操作跳过下一个结点后删除该结点。 由于题目中提到了该链表非空，所以可以确定 `q = h->next` 一定不为空。 但是如果链表中只有一个结点的话，我们还需要在删除该结点后，将尾指针指向头结点的位置，即 `if (p == q) p = h;`

考点　链表

#18 408真题

408-2023-1

答案

D

解析

线性表的顺序存储结构采用一组地址连续的存储单元依次存储线性表的数据元素。特点是逻辑上相邻的数据元素在物理位置上相邻。线性表顺序存储结构是一种随机存取的存储结构，设线性表的每个元素占 L 个存储单元，第一个元素a1的存储地址是 LOC(a1)，则任意元素ai的 LOC(ai)=LOC(a1)+(i-1)*L。因此获取第 i 个值的算法为常量阶 O(1)。

考点　顺序表

#19　408真题

408-2023-2

答案

C

解析

本题主要考察双链表的插入操作，解决这类问题可在纸上画出具体的双链表进行模拟。

执行完语句：

```
s->next = p->next;
p->next = s;
```

后，s->next 已经指向 p 原来的后继结点，同时 p->next 已经指向 s。因此，还需要完成两个指针的修改：

- s->prev 需要指向 p；
- s->next->prev 需要指向 s。

由于在执行 p->next = s 后，p->next 与 s 指向同一个结点，因此：

```
s->prev = s->next->prev;
```

等价于：

```
s->prev = p;
```

故选项 C：

```
s->prev = s->next->prev;
s->next->prev = s;
```

能够正确完成插入操作，因此本题答案为 C。

考点　链表

#20　408真题

408-2024-1

答案

D

解析

代码分解分析：

```
q = p->next;
```

现在 q 指向的是 p 的下一个结点。

```
p->next = q->next;
```

这一步将 q 从链表中“摘除”：原本是 p -> q -> q->next，现在变成了 p -> q->next，也就是说 q 不再出现在链表的原位置。

```
q->next = h->next;
```

这一步将 q->next 指向当前链表的第一个有效结点（注意是 h->next，即第一个结点，不是头结点）。

```
h->next = q;
```

把 q 接到头结点之后，也就是插入到链表头部（第一个有效结点之前）。

操作的效果是：
从链表中间删除了 q，然后把它插入到了头结点之后，也就是插入到链表第一个有效结点之前。
所以这段代码的功能是：

> 把 q 指向的结点插入到 h 的后面

正确答案选择 D。

考点　链表

#21　408真题

408-2026-1

答案

A

解析

在顺序存储结构中，元素连续存放以保持逻辑顺序。 表头插入元素时，需将所有现有元素后移一位为新元素腾出空间； 表头删除元素时，需将所有剩余元素前移一位以填补空位，这两种操作均必然导致元素移动。 而表尾插入或删除元素时，仅需在末尾进行操作，不影响其他元素的位置，因此不会产生移动次数。 故必然导致移动次数的操作是Ⅰ和Ⅱ。

考点　顺序表

#22　408真题

408-2026-2

答案

D

解析

【解析】题意澄清

- 双向链表结点结构为 [p2, p1]p1：后继指针（next）p2：需要被重新设置- p1：后继指针（next）
- p2：需要被重新设置- 目标：让每个结点的 p2 指向“p1 所指结点的直接后继”，即：cu->p2 = cu->p1->p1- 若 cu->p1 == NULL（尾结点），则不存在“p1 所指结点的直接后继”，此时应令：cu->p2 = NULL- 同时，遍历过程中必须始终推进 cu，否则会产生死循环。

逐项分析

❌ A

```
while(cu!=NULL) {
cu->p2=cu->p1->p1;
```

```
cu=cu->p1;
}
```

- 问题： 当 cu 是尾结点时，cu->p1 == NULL，此时继续访问 cu->p1->p1 会发生非法访问。

❌ B

```
while(cu!=NULL && cu->p2!=NULL) {
cu->p2 = cu->p1->p1;
cu = cu->p1;
}
```

- 问题 1：cu->p2 正是要被重新设置的指针，用它作为循环条件不合适。
- 问题 2：仍然没有判断 cu->p1 == NULL，尾结点处依然可能发生非法访问。

❌ C

```
while(cu!=NULL) {
if(cu->p1!=NULL) {
cu->p2=cu->p1->p1;
cu=cu->p1;
}
}
```

- 问题： 当 cu->p1 == NULL 时，说明 cu 到达尾结点。此时 if 语句不执行，cu 也不会更新，因此会产生死循环。

✅ D

```
while(cu!=NULL) {
if(cu->p1!=NULL) {
cu->p2=cu->p1->p1;
} else {
cu->p2=NULL;
}
cu=cu->p1;
}
```

- 对于非尾结点，令：cu->p2 = cu->p1->p1- 对于尾结点，令：cu->p2 = NULL- 每轮循环最后都执行 cu=cu->p1，保证遍历能够继续向后推进，不会死循环。

因此，正确答案为 D。

考点　链表

第三章　栈队列 10 题

#29 408真题

408-2015-1

答案

A

解析

递归调用函数时，在系统栈里保存的函数信息需满足先进后出的特点，依次调用了 `main()`，`S(1)`，`S(0)`，故栈底到栈顶的信息依次是 `main()`，`S(1)`，`S(0)`。

考点　栈

#30　408真题

408-2015-2

答案

B

解析

本题考察卡特兰数。根据二叉树前序遍历和中序遍历的递归算法中递归工作栈的状态变化得出：前序序列和中序序列的关系相当于以前序序列为入栈次序，以中序序列为出栈次序。因为前序序列和中序序列可以唯一地确定一棵二叉树，所以题意相当于“以序列 a, b, c, d 为入栈次序，则出栈序列的个数为多少“，对于 n 个不同元素进栈，出栈序列的个数n+11C2nn=14。

考点　入栈出栈序列、卡特兰数

#31　408真题

408-2017-2

答案

C

解析

I 的反例：计算斐波拉契数列迭代实现只需要一个循环即可实现。 III 的反例：入栈序列为 1、2, 进行如下操作 PUSH、PUSH、POP、POP，出栈次序为 2、1；进行如下操作 PUSH、POP、PUSH、POP，出栈次序为 1、2。 IV，栈是一种受限的线性表，只允许在一端进行操作。 因此 II 正确。

考点　栈、入栈出栈序列

#32　408真题

408-2018-1

答案

B

解析

本题考察 后序表达式求值，调用三次 F() 栈中内容如下表所示：

调用次数	S1 初始状态（栈底 → 栈顶）	S2 初始状态（栈底 → 栈顶）	弹出操作数 a, b	弹出运算符 op	执行运算 b op a	运算结果入栈后 S1 状态（栈底 → 栈顶）
第一次	5, 8, 3, 2	×, −, +	a=2, b=3	+	3 + 2 = 5	5, 8, 5
第二次	5, 8, 5	×, −	a=5, b=8	−	8 − 5 = 3	5, 3
第三次	5, 3	×	a=3, b=5	×	5 × 3 = 15	15

调用三次 F() 后，S1 的栈顶是 15，因此选择 B。

考点　后序表达式求值

#33　408真题

408-2018-2

答案

C

解析

本题考察入栈出栈序列：A 的操作顺序：①①②②①①③③。B 的操作顺序：②①①①①①③。D 的操作顺序：②②②②②①③③③③。对于 C：首先输出 3，说明 1 和 2 必须先依次入栈，而此后 2 肯定比 1 先输出，因此无法得到 1,2 的输出顺序。

考点　入栈出栈序列

#34 408真题

408-2020-2

答案

D

解析

按题意，出入栈操作的过程如下：

| 操作 | 栈内元素 | 出栈元素 || --- | --- | --- || Push | a | || Push | a b | || Pop | a | b || Push | a c | || Pop | a | c || Push | a d | || Push | a d e | || Pop | a d | e |

故出栈序列为 c,e。

考点　入栈出栈序列

#35 408真题

408-2021-2

答案

D

解析

每次入队有两个选择，但是出队只有能一个选择，所以 2 在出队时必然和 1 相邻，3 在出队时必然与 2 相邻，以此类推，可以判断 D 选项是不可能的，因为 1 和 2 没有相邻。

考点　队列

#36 408真题

408-2022-2

答案

D

解析

参考入栈出栈序列，如果给定一个入栈序列，我们可以得到其所有可能的出栈序列，所以当给定一个出栈序列时，我们可以判断其是否正确，所以 A、B、C 均为错误选项。

考点　入栈出栈序列

#37 408真题

408-2024-2

答案

A

解析

本题有两种解法：第一种是使用栈 将中序转后序。 第二种方式是先将中序表达式转换为二叉树，然后再对二叉树进行后序遍历，得到后续表达式。

这里建议使用第二种方式，对于选择题比较高效。

考点　中缀转后缀

#38 408真题

408-2025-2

答案

D

解析

栈的容量为 3，意味着在任何时候，栈中的元素数量不能超过 3。需要选择一个在某些时刻栈的深度超过 3 的表达式。通过分析各个选项的嵌套深度，可以发现：

- A. (a+[b+(c+d)e]+f)+g-h最深嵌套：([(+)])，最多 3 个未匹配括号，正确。
- 最深嵌套：([(+)])，最多 3 个未匹配括号，正确。
- B. [a*((b+c)/(d-e)+f/g)]-h最深嵌套：[(())]，最多 3 个未匹配括号，正确。
- 最深嵌套：[(())]，最多 3 个未匹配括号，正确。
- C. [a*(b-(c-d)*e/(f+g))-h]最深嵌套：[()()]，最多 3 个未匹配括号，正确。
- 最深嵌套：[()()]，最多 3 个未匹配括号，正确。
- D. [a-(b+[c*(d+e)-f]+g+h)]最深嵌套：[([])]，最多 4 个未匹配括号，错误。
- 最深嵌套：[([])]，最多 4 个未匹配括号，错误。

考点　栈

第四章　串 3 题

#44 408真题

408-2015-8

答案

C

解析

由题中失配 s[i] ≠ s[j] 时，i = j = 5", 可知题中的主串和模式串 的位序都是从 0 开始的（要注意灵活应变）。按照 next 数组生成算法，对于 t 有：

| 编号 | 0 | 1 | 2 | 3 | 4 | 5 || --- | --- | --- | --- | --- | --- | --- || t | a | b | a | a | b | b || next | -1 | 0 | 0 | 1 | 1 | 2 |

依据 KMP 算法 “当失配时，i 不变，j 回退到 next[j] 的位置并重新比较”，当失配 s[i] ≠ s[j]时，i=j=5, 由上表不难得出 next[j]=next[5]=2（位序从 0 开始）。从而最后结果应为 i=5（i 保持不变），j=2 。

考点　KMP算法

#45 408真题

408-2019-9

答案

B

解析

假设位序都是从 0 开始的，按照 next 数组生成算法，对于 S 有

| | | | | | | || --- | --- | --- | --- | --- | --- | --- || 编号 | 0 | 1 | 2 | 3 | 4 | 5 || S | a | b | a | a | b | c || next | -1 | 0 | 0 | 1 | 1 | 2 |

根据 KMP 算法，第一趟连续对比 6 次，在模式串的 5 号位和主串的 5 号位匹配失败，模式串的下一个比较位置为 next[5]，即下一次比较从模式串的 2 号位和主串 5 号位开始，然后直到模式串 5 号位和主串 8 号位匹配，第二趟比较 4 次，模式串匹配成功。单个字符的比较次数为 10 次，所以选 B。

考点　KMP算法

#46　408真题

408-2024-6

答案

A

解析

本题参考 修正后的 next 数组，传统 KMP 使用 next 数组，不过会存在 j=next[j]的情况，因此采用 next_val[] 来优化，处理好 next[] 和 next_val[] 后，举例即可，比如倒数第二个元素 a 匹配失败，会回到 -1 的位置，从 -1 的位置走到 4 的位置需要右滑的距离为 5 位。

| 下标 | 0 | 1 | 2 | 3 | 4 | 5 || --- | --- | --- | --- | --- | --- | --- || 元素 | a | a | b | a | a | b || next | -1 | 0 | 1 | 0 | 1 | 2 || next_val | -1 | -1 | 1 | -1 | -1 | 1 |

考点　KMP算法

第五章　数组广义表　6 题

#61　408真题

408-2016-4

答案

B

解析

参考三对角矩阵，采用压缩存储，将 3 条对角线上的元素按行优先方式存放在一 维数组 B 中，且a1,1存放于 B[0] 中，其存储形式如下所示：

a1,1a1,2a2,1a2,2a2,3.....an-1,nan,n-1an,n可以计算矩阵 A 中 3 条对角线上的元素ai,j(1≤i,j≤n,|i−j|≤1)在一维数组 B 中存放的下标为k=2i+j−3。

解法一：针对该题，仅需将数字逐一代入公式里面即可：k=2x30+30-3 = 87, 结果为 87。

解法二：观察上图的三对角矩阵不难发现，第一行有两个元素，剩下的在元素m30,30所在行之前的 28 行（注意下标1≤i≤100、1≤j≤100）中每行都有 3 个元素，而m30,30之前仅有一个元素m30,29, 那么不难发现元素m30,30在数组 N 中的下标是：2+28x3+2-1 = 87。

考点　特殊矩阵、三对角矩阵

#62 408真题

408-2017-3

答案

A

解析

三元组表的结点存储了行 row、列 col、值 value 三种信息，是主要用来存储稀疏矩阵的一种数据结构。十字链表将行单链表和列单链表结合起来存储稀疏矩阵。邻接矩阵空间复杂度达O(n2),不适用于存储稀疏矩阵。二叉链表又名左孩子右兄弟表示法，可用于表示树或森林。因此 A 正确。

考点　特殊矩阵

#63 408真题

408-2018-3

答案

A

解析

数组N的下标从0开始，第一个元素m1,1对应存入n0，矩阵M的第一行有 12 个元素，第二行有 11 个，第三行有 10 个，第四行有 9 个，第五行有 8 个，所以m6,6是第 12 + 11 + 10 + 9 + 8 + 1 = 51 个元素，下标应为 50, 故选 A。

考点　特殊矩阵

#64 408真题

408-2020-1

答案

C

解析

上三角矩阵 按列优先存储，先存储仅 1 个元素的第一列，再存储有 2 个元素的第二列，以此类推。加 7,2 位于左下角，对应右上角的元素为加 2, 7 , 在加 2,7 之前存有

- 第 1 列：1- 第 2 列：2- .....- 第 6 列：6- 第 7 列：1

前面共存有 1 + 2 + 3 + 4 + 5 + 6 + 1 = 2 2 个 元 素（数组下标范围为 0∽21），注意数组下标从 0 开始，故加工 7 在数组 N 中的下标为 2 2 , 即加 7,2 在数组 N 中的下标为 22。

考点　特殊矩阵

#65 408真题

408-2021-3

答案

B

解析

`A[3][3]` 和 `A[0][0]` 相差 3 行 3 列， `A[5][5]` 和 `A[3][3]` 相差 2 行 2 列，其在内存中的偏移比值为 3 / 2。 由此可以计算 `&A[5][5]`： 220 + (220 - 100) / 3 * 2 = 300

考点　二维数组

#66 408真题

408-2023-3

答案

A

解析

存储稀疏矩阵 M，三元组表的表项存储了行 row、列 col、值 value 三种信息，除此之外，我们还需要知道矩阵 M 的规模 rows × cols，即 M 的行数 rows 和 M 的列数 cols，这个信息应该直接给出。当我们需要某个位置的元素，可以先根据 M 的行数和列数判断是否越界，如果没有越界，在三元组进行查找，如果三元组没有保存对应位置的值代表矩阵中该位置的值为 0。本题答案选 A。

考点　特殊矩阵、三元组表

第六章　树 27 题

#89 408真题

408-2015-3

答案

D

解析

在哈夫曼树中，左右孩子权值之和为父结点权值。仅以分析选项 A 为例：若两个 10 分别属于两棵不同的子树，根的权值不等于其孩子的权值和，不符；若两个 10 属于同棵子树，其权值不等千其两个孩子（叶结点）的权值和，不符。B、C 选项的排除方法一样。

考点　哈夫曼树

#90 408真题

408-2015-4

答案

D

解析

只有两个结点的平衡二叉树的根结点的度为 1，A 错误。中序遍历后可以得到一个降序序列，树中最大元素一定无左子树（可能有右子树），因此不一定是叶结点，B 错误。最后插入的结点可能会导致平衡调整，而不一定是叶结点，C 错误。

考点　平衡二叉树

#91 408真题

408-2016-5

答案

C

解析

解法一：树有一个很重要的性质：在 n 个结点的树中有 n-1 条边，“那么对于每棵树，其结点数比边数多 1”。题中的森林中的结点数比边数多 10（即 25-15=10），显然共有 10 棵树。

解法二：若考生再仔细分析可发现，此题也是考察图的某些方面的性质：生成树和生成森林。此时对于图的生成树有一个重要的性质：若图中顶点数为 n，则它的生成树含有 n-1 条边。对比解法一中树的性质，不难发现两种解法都利用到了“树中结点数比边数多 1”的性质，接下来的分析如解法一。

考点　森林的概念

#92　408真题

408-2017-4

答案

B

解析

先序序列是根左右，中序序列是左根右，递归进行。如果所有非叶结点只有右子树，先序序列和中序序列都是先父结点，然后右子树，递归进行，因此 B 正确。

考点　二叉树遍历

#93　408真题

408-2017-6

答案

D

解析

哈夫曼编码是前缀编码，各个编码的前缀各不相同，因此直接拿编码序列与哈夫曼编码一一比对即可。序列可分割为 0100 011 001 001 011 11 0101，译码结果是 a f e e f g d，选项 D 正确。

考点　哈夫曼编码

#94　408真题

408-2018-4

答案

A

解析

非叶结点的度均为 2, 且所有叶结点都位于同一层的完全二叉树就是满二叉树。对一棵高度为 h 的满二叉树（空树h=0）, 其最后一层全部是叶结点，数量为2h−1; 总结点数为2h−1。因此当2h−1=k时，可以得到2h−1=2k−1。

考点　满二叉树

#95　408真题

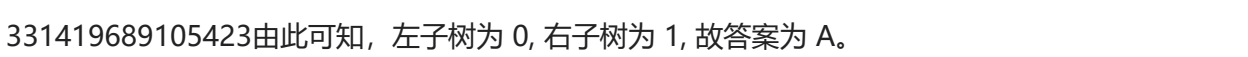
408-2018-5

答案

A

解析

构造一棵符合题意的哈夫曼树，如下图所示。



331419689105423由此可知，左子树为 0, 右子树为 1, 故答案为 A。

这题也可以 使用排除法，因为哈夫曼编码是前缀编码，所以任意编码都不能是另一个编码的前缀，根据这一点可以排除掉 B、C 选项。 D 选项也是错误的，因为字符 e 的出现次数为 10 很高，它不能哈夫曼编码长度最短，这不合理。

考点　哈夫曼树

#96 408真题

408-2019-2

答案

B

解析

本第一步，需要知道如何将一棵 树转化为二叉树：对于每一个结点，第一个孩子结点放左子树，其余孩子结点（即第一个孩子结点的兄弟结点）放在第一个孩子结点的右子树，其余的孩子结点再依次放在右子树的右子树，依次类推。

第二步，需要知道选项中的四种遍历方式先序遍历：该结点，左子树，右子树中序遍历：左子树，该结点，右子树后序遍历：左子树，右子树，该节点层次遍历：队列实现前 3 种遍历方式，左子树都是先于右子树，“先”、“中”、“后”指的是访问该结点的次序，对于上图，我们发现，对树的后序遍历与对二叉树的中序遍历相同，选 B。

考点　树和二叉树的转换

#97 408真题

408-2019-3

答案

C

解析

哈夫曼树是一颗带权路径长度最短二叉树，有性质：n 个叶子结点的哈夫曼树，共 2n-1 个结点 2n-1 = 115 解得 n = 58，选 C。

考点　哈夫曼树

#98 408真题

408-2019-4

答案

A

解析

非空平衡二叉树中插入结点，在失去平衡调整前，一定插入在叶结点的位置。

若删除的是 T1 的叶结点，则删除后平衡二叉树不会失去平衡，即不会发生调整，再插入此结点得到的二叉平衡树 T1 与 T3 相同;若删除后平衡二叉树失去平衡而发生调整，再插入结点得到的二叉平衡树 T3 与 T1 可能不同。I 正确。例如，如下图所示，删除结点 0，平衡二叉树失衡调整，再插入结点 0 后，平衡二叉树和以前不同。

对于比较绝对的说法 II 和 III，通常只需举出反例即可。

10232132130删除 0调整插入 0若删除的是 T1 的非叶结点，且删除和插入操作均没有导致平衡二叉树的调整 (这时可以首先想到删除的结点只有一个孩子的情况)，则该结点从非叶结点变成了叶结点，T1 与 T3

显然不同。例如，如下图所示，删除结点 2，用右孩子结点 3 填补，再插入结点 2，平衡二叉树和以前不同。

10231031032删除 2调整插入 0若删除的是 T1 的非叶结点，且删除和插入操作后导致了平衡二叉树的调整，则该结点有可能通过旋转后继续变成非叶结点，T1 与 T3 相同。例如，如下图所示，删除结点 2，用右孩子结点 3 填补，再插入结点 2，平衡二叉树失衡调整，调整后的平衡二叉树和以前相同。

21331312删除 2调整插入 2调整213

考点　平衡二叉树

#99　408真题

408-2020-3

答案

A

解析

二叉树采用顺序存储时，用数组下标来表示结点之间的父子关系。对于一棵高度为 5 的二叉树，为了满足任意性，其 1~5 层的所有结点都要被存储起来，即考虑为一棵高度为 5 的满二叉树，总共需要存储单元的数量为 1 + 2 + 4 + 8 + 16 = 31。

考点　二叉树存储

#100　408真题

408-2020-4

答案

C

解析

本题考察的是 森林的遍历，森林 F 的先根遍历序列对应其二叉树 T 的先序遍历序列，森林 F 的中根遍历序列对应其二叉树 T 的中序遍历序列。即 T 的先序遍历序列为 a,b,c,d,e,f，中序遍历序列为 b,a,d,f,e,c。根据二叉树 T 的先序序列和中序序列可以唯一确定它的结构，构造过程如下：

abdfecabcdfcabcdfeabcdef可以得到二叉树 T 的后序序列为 b,f,e,d,c,a。

考点　二叉树遍历、二叉树构建

#101　408真题

408-2021-4

答案

C

解析

这题考察的是两点，一是二叉树构建，即根据 先序 和 中序 构建二叉树。二是森林转二叉树，构建出二叉树后即可反向转化得到对应的森林。 森林中树的个数 即 二叉树根结点的右结点个数之和。

考点　二叉树构建、二叉树和森林的转换

#102　408真题

408-2021-5

答案

B

解析

根据 Huffman 构建过程 来构建得到如下二叉树：

```
89
/      \
52      37
/   \    /  \
22   30  16  21
/  \
10  12
```

带权路径长度 为： (10+12)×3+(30+16+21)×2=200

考点　哈夫曼树

#103　408真题

408-2021-9

答案

A

解析

保证特性中的结点数量尽量多，然后每个结点中元素数量尽量多。 3 阶 B 树 中个结点最多有两个元素，三个孩子。

考点　树的概念

#104　408真题

408-2022-3

答案

B

解析

对于此类题，每种情况只需举出一个反例即可。如图 1 所示，q 是 p 的双亲，中序遍历序列为 {p, q}，I 可能。如图 2 所示，q 是 p 的右孩子，中序遍历序列为 {p, q}，II可能。 如图 4 所示，q 是 p 的双亲的双亲，中序遍历序列为 {x, p, q}，IV 可能。如图 3 所示，q 是 p的右兄弟，F 是 q 和 p 的父结点，中序遍历要求先遍历左子树，再访问根结点，最后遍历右子树，因此一定先访问 p，再访问 F，最后访问 q，p 和 q 不可能相邻出现，II 不可能。

pqpqFpqqxp图 1图 2图 3图 4

考点　二叉树遍历

#105　408真题

408-2022-4

答案

C

解析

高度为n的二叉树最多有1+2+⋯+2n−1=2n−1个结点，高度为n的三叉树最多有f(n)=1+3+⋯+3n−1=3−13n−1−23n−1。f(5)=121f(6)=364因为121<244<364，所以高度至少为 6。

考点　树的概念

#106 408真题

408-2022-5

答案

D

解析

可以画一个简单的特例来证明。图 1 是满足条件的二叉树 T1，图 2 是满足条件的二叉树 T2，结点中有值表示这个结点是编码字符。T1 和 T2 的结点数不同，A 错误。T1 的高度等于 T2 的高度，B 错误。出现频次不同的字符在 T1 中也可能处于相同的层，C 错误。对于定长编码码集，所有字符一定都在 T2 中处于相同的层，而且都是叶子结点。

312123图1图2

考点　哈夫曼树

#107 408真题

408-2023-4

答案

B

解析

计算字符集 S 构造的 哈夫曼编码 的 加权路径长度，我们需要使用字符的频次来确定每个字符的编码长度，并计算加权平均值。给定字符集 S 中各字符出现的频次为 3，4，5，6，8，10，我们可以按照哈夫曼编码算法构造哈夫曼树。构建的哈夫曼树如图所示。加权平均长度 =（编码长度 1 × 频次 1 + 编码长度 2 × 频次 2 + … + 编码长度 n × 频次 n）/（频次 1 + 频次 2 + … + 频次 n），在本题中，加权平均长度=((3+4+5+6)×3 + (8+10)×2)/(3+4+5+6+8+10)=2.5。本题答案选 B。

3615211110783456

考点　哈夫曼树

#108 408真题

408-2024-3

答案

A

解析

根据中序遍历结果和 v 是子树的根节点这些信息，则可以判定 p,q 分别在 v 的的左右子树上，对于左子树而言，根据中序遍历结果为...p，则没有右孩子，同理，q 没有左孩子。或者假设 p 有右孩子,q 有左孩子，则中序遍历结果中 p,v 之间一定还有序列，v,q 之间也一定还有序列，和题意冲突。

考点　二叉树遍历

#109 408真题

408-2025-3

答案

D

解析

在二叉树的顺序存储表示中，数组中的每个位置对应二叉树中特定节点。若节点值非负（-1 表示不存在），则非 -1 节点的父节点必须存在（根节点除外）。对于选项 D，索引 8 处的节点值为 19，其父节点索引为 3（由公式⌊(8−1)/2⌋=3计算），但索引 3 处的值为 -1，即父节点不存在，违反了二叉树顺序存储的规则。因此，选项 D 不能表示一棵二叉树。

〔图〕

其他选项均满足每个非 -1 节点（除根节点）都有存在的父节点，是有效的二叉树顺序存储表示。

考点　二叉树存储

#110　408真题

408-2025-4

答案

B

解析

完全二叉树中，度为 1 的结点可能存在。比如一颗完全二叉树只有两个结点，那么根结点的度就是 1。A 选项错误。

森林转二叉树 有固定的方法，B 选项正确。

考点　二叉树和森林的转换

#111　408真题

408-2025-5

答案

D

解析

按照 哈夫曼树 构造：频次为 2, 3, 4, 6, 8, 10, 11

1. 合并 2 + 3 = 52. 合并 4 + 5 = 93. 合并 6 + 8 = 144. 合并 9 + 10 = 195. 合并 11 + 14 = 256. 合并 19 + 25 = 44 （根）

可以得到以下二叉树：

```
44
/    \
19      25
/  \    /  \
9    10 11  14
/ \          / \
4   5        6   8
/ \
2   3
```

编码长度不小于 3 的字符：2, 3, 4, 6, 8。一共有 5 个字符编码长度不小于 3，正确答案为 D。

考点　哈夫曼树

#112　408真题

408-2026-3

答案

C

解析

【解析】首先，根据层序遍历序列 a,b,g,c,d,e,f可知根节点为a。结合中序遍历b,e,d,f,c,a,g，确定左子树包含节点b,e,d,f,c，右子树仅包含g。

左子树的层序序列为b,c,d,e,f，中序序列为b,e,d,f,c，因此左子树的根为b。由于b在中序中为首，故无左子树，其右子树的根为层序中下一个节点c。

对于以c为根的子树，中序为e,d,f,c，故c无右子树，其左子树的根为层序中的d。对于以d为根的子树，中序为e,d,f，故d的左子节点为e，右子节点为f。

因此树的结构为：

- a的左子节点为b，右子节点为g；
- b的左子节点为空，右子节点为c；
- c的左子节点为d，右子节点为空；
- d的左子节点为e，右子节点为f。

后序遍历顺序为：左子树的后序、右子树的后序、根节点。左子树的后序依次为e,f,d,c,b，右子树的后序为g，根为a，故后序遍历序列为e,f,d,c,b,g,a，对应选项 C。

考点　二叉树遍历、二叉树构建

#113　408真题

408-2026-4

答案

B

解析

【解析】

这是一道 森林转二叉树 的经典题。

关键结论（必须掌握）

设森林中第i棵树对应二叉树的高度为hi，则森林转换为二叉树后：

二叉树高度=imax{hi+(i−1)}其中：

- 第i棵树表示森林中从左到右的第i棵树；
- (i−1) 来自各棵树根结点之间形成的右兄弟链；
- 为了使最终高度最小，应将高度较大的树排在前面。

先计算每棵树对应二叉树的最小高度

对于一棵有n个结点的普通树，采用左孩子-右兄弟表示法转换为二叉树后，由于根结点没有右孩子，因此最小高度为：

hmin=1+⌈log2n⌉各棵树对应二叉树的最小高度如下：

| 结点数 | 最小高度 || --- | --- || 7 | 1+⌈log27⌉=4 || 5 | 1+⌈log25⌉=4 || 4 | 1+⌈log24⌉=3 || 3 | 1+⌈log23⌉=3 || 2 | 1+⌈log22⌉=2 |

按最优顺序（高度从大到小）排列：

4,4,3,3,2---

计算最终二叉树高度

| i | 树高hi | hi+(i−1) || --- | --- | --- || 1 | 4 | 4 || 2 | 4 | 5 || 3 | 3 | 5 || 4 | 3 | 6 || 5 | 2 | 6 |

最大值为:

6因此答案为 B。

考点　二叉树和森林的转换

#114 408真题

408-2026-5

答案

D

解析

【解析】为了最小化带权路径长度（WPL），需构建哈夫曼树。节点权值依次为 1, 2, 4, 5, 8, 10, 12。构建过程如下:

1. 合并权值 1 和 2，得到新节点 3;
2. 合并 3 和 4，得到新节点 7;
3. 合并 5 和 7，得到新节点 12;
4. 合并 8 和 10，得到新节点 18;
5. 合并原始权值 12（节点 g）与内部节点 12，得到新节点 24;
6. 最后合并 18 和 24，得到根节点 42。

由此树结构可知，节点 e（权值 8）深度为 2（路径长度），同时节点 f（权值 10）和 g（权值 12）深度也为 2，而其他节点深度均不同（d 深度为 3，c 深度为 4，a、b 深度为 5）。因此，与节点 e 处于相同深度的节点是 f 和 g。

考点　哈夫曼树、带权路径长度

#115 408真题

408-2026-8

答案

D

解析

【解析】平衡二叉树高度为 4（根节点高度为 1），因此左右子树的高度组合为:

- 两者均为 3，或- 一个为 3、另一个为 2。

为最大化节点数之差，应使较高子树取最大节点数，较低子树取最小节点数。

- 高度 3 的平衡二叉树最大节点数为 7（满二叉树），- 高度 2 的最小节点数为 2（根节点加一个子节点），此时节点数之差为7−2=5。

若左右子树高度均为 3，节点数之差最大为7−4=3。因此，根节点的左右子树节点数之差最多为 5。

考点　平衡二叉树

第七章　图 28 题

#123 408真题

408-2015-5

答案

D

解析

画出该有向图图形如下：

0123采用图的深度优先遍历，共 5 种可能： , , , , , 选 D。

考点　图的遍历

#124 408真题

408-2015-6

答案

C

解析

V4开始，kruskal 算法选中的第一条边一定是权值最小的 (V1,V4), B 错误。由于V1和V4已经可达，第二条边含有V1和V4的权值为 8 的一定符合prim 算法，排除 A、D。

考点　最小生成树

#125 408真题

408-2016-3

答案

C

解析

在确保队列先进先出原则的前提下。根据题意具体分析：入队顺序为 8, 4, 2, 5, 3, 9, 1, 6, 7，出队顺序为 1~9。入口和出口之间有多个队列（n 条轨道），且每个队列（轨道）可容纳多个元素（多列列车）。如此分析：显然先入队的元素必须小千后入队的元素（如果 8 和 4 入同一队列，8 在前 4 在后，那么出队时只能是 8 在前 4 在后），这样 8 入队列 1，4 入队列 2, 2 入队列 3，5 入队列 2（按照前面的原则“大的元素在小的元素后面”也可以将 5 入队列 3，但这时剩下的元素 3 就必须放到一个新的队列里面，无法确保”至少“，本应该是将 5 入队列 2，再将 3 入队列 3，不增加新队列的情况下，可以满足题意“至少”的要求），3 入队列 3，9 入队列 1，这时共占了 3 个队列，后面还有元素 1，直接再占用一个新的队列 4，1 从队列 4 出队后，剩下的元素 6 和 7 或者入队到队列 2 或者入队到队列 3（为简单起见我们不妨设 n 个队列的序号分别为 1, 2, ..., n），这样就可以满足题目的要求。综上，共占用了 4 个队列。当然还有其他的入队出队的情况，请考生们自己推演。但要确保满足：O 队列中后面的元素大千前面的元素；@确保占用最少（即满足题目中的“至少") 的队列。

考点　队列

#126 408真题

408-2016-6

答案

D

解析

对于本题，只需按深度优先遍历的策略进行遍历即可。对千选项 A: 先访问V1, 然后访问与V1邻接且未被访问的任一顶点（满足的有V2、V3和V5），此时访问V5, 然后从V5出发，访问与V5邻接且未被访问的任一顶点（满足的只有V4），然后从V4出发，访问与V4邻接且未被访问的任一顶点（满足的只有V3），然后从V3出发，访问与V3邻接且未被访问的任一顶点（满足的只有V2），结束遍历。选项 B 和 C 的分析方法与选项 A 相同，不再赘述。对千选项 D, 首先访问V1, 然后从V1出发，访问与V1邻接且未被访问的任一顶点（满足的有V2、V3和V5）,然后从V2出发，访问与V2邻接且未被访问的任一顶点（满足的只有V5），按规则本应该访问V5, 但选项 D 却访问V3, 因此 D 错误。

考点　图的遍历

#127　408真题

408-2016-7

答案

B

解析

根据拓扑排序的规则，输出每个顶点的同时还要删除以它为起点的边，这样对各顶点和边都要进行遍历，故拓扑排序的时间复杂度为O(n+e)。

考点　复杂度分析、邻接表、拓扑排序

#128　408真题

408-2016-8

答案

B

解析

根据 Dijkstra 算法，从顶点 1 到其余各顶点的最短路径如下表所示。

| 顶点 | 第 1 趟 | 第 2 趟 | 第 3 趟 | 第 4 趟 | 第 5 趟 || --- | --- | --- | --- | --- | --- || 2 | 5v1→v2 | 5v1→v2 | | | || 3 | ∞ | ∞ | 7v1→v2→v3 | | || 4 | ∞ | 11v1→v5→v4 | 11v1→v5→v4 | 11 v1→v5→v4 | 11v1→v5→v4 || 5 | 4v1→v5 | | | | || 6 | ∞ | 9v1→v5→v6 | 9v1→v5→v6 | 9v1→v5→v6 | || 集合 S | {1, 5} | {1, 5, 2} | {1, 5, 2, 3} | {1, 5, 2, 3, 6} | {1, 5, 2, 3, 6, 4} |

考点　最短路径

#129　408真题

408-2017-5

答案

B

解析

后序序列是先左子树，接着右子树，最后父结点，递归进行。根结点左子树的叶结点首先被访问，它是 e。接下来是它的父结点 a, 然后是 a 的父结点 c。接着访问根结点的右子树。它的叶结点 b 首先被访问，然后是 b 的父结点 d, 再者是 d 的父结点 g。最后是根结点 f。因此 d 与 a 同层，B 正确。

考点　二叉树遍历

#130　408真题

408-2017-7

答案

B

解析

无向图边数的两倍等于各顶点度数的总和。由于其他顶点的度均小于 3，可以设它们的度都为 2，设它们的数量是 X，可列出方程 4x3 + 3x4 + 2x = 16x2，解得 x = 3。4 + 3 + 3 = 11，B 正确。

考点　图的概念

#131　408真题

408-2018-6

答案

C

解析

根据二叉排序树的特性：中序遍历（LNR）得到的是一个递增序列。图中二叉排序树的中序遍历序列为 xi,x3,x5,x4,x2, 可知x3<x5<x4。

考点　二叉排序树

#132　408真题

408-2018-7

答案

D

解析

拓扑排序每次选取入度为 0 的结点输出，经观察不难发现拓扑序列前两位一定是 1,5 或 5,1（因为只有 1 和 5 的入度均为 0，且其他结点都不满足仅有 1 或仅有 5 作为前驱）。因此 D 显然错误。

考点　拓扑排序

#133　408真题

408-2019-5

答案

C

解析

AOE 网是以边表示活动的有向无环网。活动 d 开始必须满足活动 a 和活动 b 结束，所以最早开始时间是 12。最晚开始时间是指不会延长整个网的结束时间的前提下，d 的开始时间。结点 1 到结点 6 的最长路径（关键路径）是 27，结点 4 的最晚开始时间是 27-6=21，结点 d 的最晚开始时间是 21-7=14。答案选 C。

考点　关键路径

#134　408真题

408-2019-6

答案

A

解析

先将该表达式转换成有向二叉树，注意到该二叉树中有些顶点时重复的，为了节省存储空间，可以去除重复的顶点（使顶点个数达到最少），将有向二叉树去重转换成有向无环图，结果如图所示：

*+/xy

考点　图表示树

#135　408真题

408-2020-6

答案

B

解析

DFS是一个递归算法，在遍历过程中，先访问的顶点被压入栈底。设在图中有顶点vi，它有后继顶点vj，即存在边。根据 DFS 的规则，vi入栈后，必先遍历完其后继顶点后vj才会出栈，也就是说vi会在vj之后出栈，在如题所指的过程中，vi在vj后打印。由于vi和vj具有任意性，从上面的规律可以看出，输出顶点的序列是逆拓扑有序序列。

考点　图的遍历

#136　408真题

408-2020-7

答案

A

解析

kruskal 算法 依次将当前 不会导致环路的最短边 加入最小生成树中：

acbdef1220186571410911acbdef1220186571410911acbdef1220186571410911acbdef12186571410911acbdef12186571410911第一步选取 < b, f >第二步选取 < b, d >第三步选取 < a, e >第四步选取 < c, e >第五步选取 < b, e >

考点　最小生成树

#137　408真题

408-2020-8

答案

B

解析

关键路径是指权值之和最大而非边数最多的路径，故选项 A 错误。选项 B 正确，是关键路径的概念。无论是存在一条还是存在多条关键路径，增加任一关键活动的时间都会延长工程的工期，因为关键路径始终是权值之和最大的那条路径，选项 C 错误。仅有一条关键路径时，减少关键活动的时间会缩短工程的工期；存在多条关键路径时，缩短一条关键活动的时间不一定会缩短工程的工期，缩短了路径长度的那条关键路径不一定还是关键路径，选项 D 错误。

考点　关键路径

#138　408真题

408-2021-6

答案

D

解析

根据 AVL 旋转方法 可知，这题采用 RL 型旋转，旋转后树的结构为：

```
25
/  \
20   30
/  \    \
16  23    40
```

根结点为 25

考点　平衡二叉树

#139　408真题

408-2021-7

答案

A

解析

依次选择入度为 0 的顶点进行输出，得到拓扑序列，发现只有一种可能性。

考点　拓扑排序

#140　408真题

408-2021-8

答案

C

解析

在dijkstra 算法中，每次在 dist 数组中选择一个最小值加入顶点集，并从该顶点出发修改 dist 数组。本题中 dist 数组的变化过程如下：`dist[26, 3, ∞, 6] → [25, 3, ∞, 6] → [21, 3, 14, 6]`

考点　最短路径

#141　408真题

408-2022-6

答案

D

解析

注意，E 是图的边数，V 是图的顶点数。A 和 B 明显错误，如图 1 所示，|V| < |E|，但图 G 不连通；如图 2 所示，|V| > |E|，但图 G 不连通。如图 3 所示，在无向图中至少要有 |V| - 1 个顶点才可能连通，顶点数小于 |V| - 1 一定不可能连通，C 错误，D 正确。

图 1图 2图 3

考点　图的概念

#142　408真题

408-2022-7

答案

B

解析

在 AOE 网中，活动的时间余量 = 结束顶点的最迟开始时间 - 开始顶点的最早开始时间 - 该活动的持续时间。根据关键路径算法得到下表：

结点编号	1	2	3	4	5	6
最早开始时间 ve(i)	0	2	5	8	9	12
最迟开始时间 vl(i)	0	4	5	8	11	12

- c 的时间余量 = vl(3) - ve(2) - 1 = 5 - 2 - 1 = 2- g 的时间余量 = vl(6) - ve(3) - 1 = 12 - 5 - 1 = 6，- h 的时间余量 = vl(5) - ve(4) - 1 = 11 - 8 - 1 = 2- j 的时间余量 = vl(6) - ve(5) - 1 = 12 - 9 - 1 = 2，

时间余量最大的活动是 g。

考点　关键路径

#143　408真题

408-2022-8

答案

D

解析

参考 B 树插入操作，在 5 阶 B 树中，除根结点外的非叶子结点的关键字数 k 需要满足 $2 \le k \le 4$。当被删关键字 x 不在终端结点（最底层非叶子结点）时，可以用 x 的前驱（或后继）关键字 y 来替代 x，然后在相应结点中删除 y。

情况①：删除 260，将其前驱 110 放入 260 处，删除 110 后的结点 不满足 5 阶 B 树定义，从左兄弟中借 85，将 85 放入根中，将根中的 90 移入结点 变为 <90, 100>。

情况②：删除 260，将其后继 280 放入 260 处，结点 不满足 5 阶 B 树定义且左右兄弟都不够借，结点可以和左兄弟 <100, 110> 以及关键字 280 合并成一个新的结点 <100, 110, 280, 300>。

情况③：在情况②中，结点 也可以和右兄弟 <400, 500> 以及关键字 350 合并成一个新的结点 <300, 350, 400, 500>。综上，T1 根结点中的关键字序列可能是 <60, 85, 110, 350> 或 <60, 90, 350> 或 <60, 90, 280>，仅 D 不可能。

考点　B树

#144　408真题

408-2023-5

答案

A

解析

如下图所示。对于后序序列 fdbeca，a 为树节点的根，因此在序号 1 中，a 首先进行绘制。同时，a 节点的左子树有 4 个节点，右子树有 1 个节点，因此 fdbe 属于左子树，c 节点属于右子树，所以我们在序号 2 的树中，填充 c。a 结点左子树的后序遍历序列为 fdbe，代表 e 为左子树的根节点，因此在序号 3 的树中，填充 e。同理 e 节点的左子树有两个节点，右子树有一个节点，因此 fdb 属于左子树，e 属于右子树，在序号 4 的树中，我们填写 b。e 的左子树的后序遍历序列为 fd，则 d 为子树的根节点，因此在

序号 5 的树中，我们填充 d，最后在序号 6 的图中，填充 f。先序序列为 a，e，d，f，b，c。本题答案选 A。

aacaecaebcaedbcaedbcf123456

考点　二叉树遍历

#145 408真题

408-2023-6

答案

B

解析

普里姆算法和克鲁斯卡尔算法是用于解决最小生成树问题的算法，它们并不直接适用于求解最短路径问题。广度优先遍历 (BFS) 是一种图遍历算法，可以用于求解单源最短路径问题。在权值均为 1 的情况下，BFS 算法从给定的起始顶点开始遍历，按照层级顺序逐步扩展，每次扩展一层。这样，当 BFS 遍历到目标顶点时，它所经过的边数就是从起始顶点到目标顶点的最短路径长度。因此，BFS 算法可以用于求解图 G 中从某个顶点到其余各个顶点的最短路径。本题答案选 B。

考点　最小生成树

#146 408真题

408-2024-4

答案

B

解析

根据邻接多重表还原图，由下图可知，b 的度为 2，d 的图为 4。

abecd

考点　邻接多重表

#147 408真题

408-2024-7

答案

D

解析

根据二叉搜索树的性质，K2的左子树的节点关键字均小于K2，可以得出K3K3，X是T中节点，由此可以得出K3<X<K2。

考点　二叉排序树

#148 408真题

408-2025-6

答案

C

解析

逐个选项来看：

- A. 错误，考虑一个有向图的环，每个顶点的入度都为 1。
- B. 错误，拓扑排序不一定是唯一的，在计算拓扑排序序列的过程中，如果有多个入度为 0 的顶点，则拓扑序列的数量大于 1。
- C. 正确，无环的图至少有一个顶点的度为 1，如果所有顶点的度都大于 1 了，那图中肯定存在环。
- D. 错误，BFS 不能求 带权图 的最短路径，求最短路径是使用 Djkstra 算法。

考点　图的概念

#149　408真题

408-2026-6

答案

D

解析

【解析】在邻接表存储中，求某点的入度需要检查所有顶点的出边链表，统计指向该点的边数。这需要访问所有|V|个顶点以及所有|E|条边，因此时间复杂度为O(|V|+|E|)。由于O(|V|+|E|)与O(max(|V|,|E|))等价，故选项 D 正确。其他选项均不能完整描述该时间复杂度。

考点　邻接表、复杂度分析

#150　408真题

408-2026-7

答案

B

解析

【解析】对于选项 A：若图 G 无环，则任意路径不能重复经过顶点，否则会形成环，因此最长路径的边数不超过 n−1。由于图是有限的，所有可能的路径数量有限，每条路径对应一个字符串（可能重复），但字符串集合 S 由有限个字符串组成，故 S 是有限集。A 正确。

对于选项 B：若图 G 无环，则任意路径最多经过n个不同的顶点，因此边数最多为n−1，对应的字符串长度最多为n−1。所以 S 中不可能存在长度为n的字符串。B 错误。

对于选项 C：若图 G 有环，则存在一个环，可以从环上某点出发沿环行走任意多圈，得到任意长的路径，从而产生长度大于n的字符串。C 正确。

对于选项 D：若图 G 有环，S 中至少包含空字符串（长度为0），而0<2n（n≥1），因此存在长度小于2n的字符串。D 正确。

综上，说法错误的是 B。

考点　图的概念

第八章　查找 18 题

#159　408真题

408-2015-7

答案

A

解析

画出查找路径图，因为折半查找的判定树是一棵二叉排序树，看其是否满足二叉排序树的要求。

5002004501805004502001801805002004501802005004 50显然，选项 A 的查找路径不满足。

考点　数组查找

#160　408真题

408-2016-9

答案

B

解析

此题为送分题。该程序采用跳跃式的顺利查找法查找升序数组中的 X, 显然是 x 越靠前，比较次数才会越少。

考点　数组查找

#161　408真题

408-2016-10

答案

A

解析

由于B+ 树的所有叶结点中包含了全部的关键字信息，且叶结点本身依关键字从小到大顺序链接，可以进行顺序查找，而 B 树不支持顺序查找（只支持多路查找）。

考点　B树、B+树

#162　408真题

408-2017-8

答案

A

解析

折半查找判定树 实际上是一棵 二叉排序树，它的中序序列是一个有序序列。可以在树结点上依次填上相应的元素，判断哪颗树符合折半查找的规则。

52140387965214038697104103275864213076958A.B.C.D.折半查找树由于其中序遍历是一个升序序列，因此相比于在以往的序列中进行二分查找，折半查找二叉树的优点在于不用自己去找中点，而是直接将要查找的关键字与根节点相比，小的话再和根节点的左子节点（又是相应的左子树中点，更加方便）比较，大的话则是和根节点的右子节点比较。

而对于这道题的解题思路就在于向上或向下取整的问题，也就是说如果升序序列是偶数个，那么中点应该偏左多右少还是左少右多。但是很显然应该进行一个统一，像 B 和 C 选项中间的对称部分明显就是选择了不同的策略。D 则由根节点左子树 4 个节点而点右子树 5 个节点可以确定用的是向下取整策略，但是我们再看它的左子节点在左子树中对应的中点左边 2 个数，右边一个数，明显是向上取整策略，策略没有统一，所以是错的。

考点　二叉排序树

#163　408真题

408-2017-9

答案

B

解析

B+ 树是应文件系统所需而产生的 B 树 的变形，前者比后者更加适用于实际应用中的操作系统的文件索引和数据库索引，因为前者磁盘读写代价更低，查询效率更加稳定。编译器中的词法分析使用有穷自动机和语法树。网络中的路由表快速查找主要靠高速缓存、路由表压缩技术和快速查找算法。系统一 般使用空闲空间链表管理磁盘空闲块。所以 B 正确。

考点　B+树

#164　408真题

408-2018-8

答案

B

解析

m阶B 树的基本性质：根结点以外的非叶结点最少含有⌈m/2⌉−1个关键字，代入m=3得到每个非叶结点中最少包含 1 个关键字，而根结点含有 1 个关键字，因此所有非叶结点都有两个孩子。此时其树形与 h=5 的满二叉树相同，可求得关键字最少为 31 个。

考点　B树

#165　408真题

408-2018-9

答案

C

解析

可以构造得到如下的 HT：

| 下标 | 0 | 1 | 2 | 3 | 4 | 5 | 6 || --- | --- | --- | --- | --- | --- | --- | --- || 关键词 | | 22 | 43 | 15 | | | |

成功时的平均查找长度 = (1+2+3)/3 = 2。

考点　散列表、平均查找长度

#166　408真题

408-2019-8

答案

C

解析

构造散列表只有当遇到关键字为空的地址时才会查找失败，key%7 之后，初始地址只可能在 0~6，所以即 0~6 到空地址的距离求平均，即为查找失败的平均查找长度初始地址是 0 的失败查找长度为 9，同理得初始地址为 1，2，3，4，5，6 的失败查找长度为 8，7，6，5，4，3，(9+8+7+6+5+4+3)/7 = 6 答案是 C。

考点　散列表、平均查找长度

#167　408真题

408-2020-5

答案

B

解析

选项 B 生成二叉排序树的过程如下，显然 B 选项错误：

445451451245123

考点　二叉排序树

#168 408真题

408-2020-10

答案

B

解析

一个 4 阶 B 树的任意非叶结点至多含有加 4-1=3 个关键字，在关键字依次插入的过程中，会导致结点的不断分裂，插入过程如下所示。

5　6　95　6　9　139　13568　9　132　568　9　12　132　5612　132　56　9812　13　152　56　98插入 5, 6, 9插入 13分裂插入 8, 2插入 12分裂插入 15得到根结点包含的关键字为 6, 9。

考点　B树

#169 408真题

408-2022-9

答案

D

解析

三者都会影响：装填因子越大，说明哈希表中存储的元素越满，发生冲突的可能性越大，平均查找长度也越大。散列函数、突解决策略会影响发生冲突的可能性。

考点　散列表

#170 408真题

408-2023-7

答案

B

解析

在 B 树中，插入 操作可能导致节点分裂。如果根节点已满，插入新键会触发根节点分裂，生成一个新的根节点，从而增加树的高度。因此，插入操作确实可能（但不一定）增加树的高度。选项 I 正确。

删除 操作可能涉及以下情况：

- 删除的键在叶子结点：直接从叶子结点移除该键，导致叶子结点内容变化。
- 删除的键在非叶子结点：B 树会用前驱或后继键（通常位于叶子结点）替换该键，然后删除前驱或后继键。这仍然会导致叶子结点的变化。
- 删除后触发合并或借键：如果删除键后某个节点键数低于最小要求（通常 $\lceil m / 2 \rceil - 1$），可能需要从兄弟节点借键或与兄弟节点合并。这些操作可能涉及叶子结点（例如，合并两个叶子结点或调

整叶子结点的键）。

在所有删除场景中，无论是直接删除键还是通过替换、合并、借键，操作最终都会影响叶子结点（键的移除或调整发生在叶子结点）。选项 II 正确。

查找关键字并不一定需要查找到叶子结点。例如，如果目标键位于根节点或中间节点，查找会在这些非叶子结点停止。选项 III 错误。

新关键字在 B 树中开始总是被插入到叶子结点中，但是可能随着分裂上溢到非叶子结点，选项 IV 错误。

考点　B树

#171 408真题

408-2023-8

答案

B

解析

在含有 600 个元素的有序表进行折半查找时，其关键字比较次数最多的情况发生在目标元素不在表中的情况下。在这种情况下，折半查找会进行到最后一步，直到左边界和右边界相遇。因此，关键字比较次数最多的情况是在查找过程中遍历整个表的情况。在进行折半查找时，每次比较会将查找范围缩小一半，直到找到目标元素或确定目标元素不在表中。对于 600个元素的有序表，因为log2(600)≈9.22，采用 9 次遍历并不能查找完成，因此需要 10 次查找。 本题答案选 B。

考点　数组查找

#172 408真题

408-2023-9

答案

C

解析

线性探测再散列法中删除一个关键字会导致后面的关键字无法通过线性探测找到正确的位置。当删除一个关键字时，为了保持散列表的连续性，通常会将后续的关键字向前移动填充空缺，这样后续的查找操作才能继续正确地找到它们。然而，如果删除的是一个位于中间位置的关键字，后面的关键字需要依次向前移动，这会导致删除操作的时间复杂度较高，因为需要移动大量的关键字。为了解决删除操作中的位置依赖性问题，可以使用删除标记来表示一个位置上的关键字已被删除。

如下表所示，查找失败的平均查找长度为 (1+3+2+1+2)/5=1.8。本题答案选 C。

| 地址 | 0 | 1 | 2 | 3 | 4 || --- | --- | --- | --- | --- | --- || Key | | 2022 | 12 | | 25 (delete) || 查找失败次数 | 1 | 3 | 2 | 1 | 2 |

考点　散列表、平均查找长度

#173 408真题

408-2024-5

答案

D

解析

折半查找有两个前提：元素有序 且 支持随机存取。四项各缺其一，因此都不适合直接使用折半查找：

- I 有序链表：有序，但链式存储不能随机存取，无法在 O(1) 时间内定位中间元素。
- II 无序数组：能随机存取，但不满足有序。
- III 有序静态链表：静态链表虽以数组作为存储池，但逻辑次序由 cur 游标串联，下标相邻不等于逻辑相邻，仍无法按序号折半定位。
- IV 无序静态链表：两个条件都不满足。

故选 D。

考点　数组查找

#174 408真题

408-2025-7

答案

C

解析

设块大小为m，块数为mn。查找复杂度为O(n/m+m) 。

根据高中数学的知识可以得知，m=n=20时，查找复杂度最低，所以m=20。

考点　数组查找、分块查找

#175 408真题

408-2025-8

答案

C

解析

- 树高为 3：每个节点内关键字个数最少取 1 时，B 树 高度为 3（类似满二叉树）——仅一种结构；
- 树高为 2:根节点关键字个数取 1，第二层两节点关键字个数均取 3——一种结构；根节点关键字个数取 2，则第二层内三个节点关键字个数分别可取 221、212、122、311、131、113 共计六种结构；根节点关键字个数取 3，则第二层显然只有四个节点各含一个关键字此一种结构。
- 根节点关键字个数取 1，第二层两节点关键字个数均取 3——一种结构；
- 根节点关键字个数取 2，则第二层内三个节点关键字个数分别可取 221、212、122、311、131、113 共计六种结构；
- 根节点关键字个数取 3，则第二层显然只有四个节点各含一个关键字此一种结构。

综上，共计 9 种可能的结构。

考点　B树

#176 408真题

408-2025-9

答案

A

解析

在散列（哈希）方法中，同义词 是指不同的元素通过哈希函数映射到同一个哈希值（或哈希地址)。这意味着这些元素在散列表中会发生冲突，因为它们试图占用相同的位置。

非同义词 指的是通过哈希函数映射到不同哈希值的元素，它们通常不会在初始哈希地址上发生冲突。

对于题目中的选项：

A. 只要散列表不满，线性探查再散列一定能找到一个空闲位置。

线性探查是在发生冲突时，按顺序查找下一个可用位置。例如，如果发生冲突，就逐一检查下一个位置，直到找到空闲位置。只要散列表有空位，线性探查一定能找到一个空闲位置。因此，这个选项是正确的。

B. 只要散列表不满，二次探查再散列一定能找到一个空闲位置。

二次探查是通过二次函数（如平方）来决定下一个检查的位置。虽然理论上在某些情况下可能会更快找到空位，但由于散列表的装填因子和特定的探查序列，可能会有探查不到空闲位置的情况，因此这个选项不一定总是正确。

C. 线性探查再散列处理的冲突，一定是发生在同义词之间。

线性探查处理的冲突可能发生在同义词之间（即初始哈希值相同的元素），但也可能发生在非同义词之间（即由于探查过程而导致的冲突）。因此，这个选项是不正确的。

D. 二次探查再散列处理的冲突，一定是发生在非同义词之间。

二次探查可能会导致同义词和非同义词之间的冲突。因此，这个选项是不正确的。

考点　散列表

第九章　排序 24 题

#192 408真题

408-2015-9

答案

C

解析

基数排序的元素移动次数与关键字的初始排列次序无关，而其他三种排序都是与关键字的初始排列明显相关的。

考点　排序算法、基数排序

#193 408真题

408-2015-10

答案

C

解析

删除 8 后，将 12 移动到堆顶，第一 次是 15 和 10 比较，第二次是 10 和 12 比较并交换，第三次还需比较 12 和 16, 故比较次数为 3 次。

8151021341612121510213416101512213416

考点　堆的概念

#194 408真题

408-2015-11

答案

A

解析

希尔排序的思想是：先将待排元素序列分割成若干子序列（由相隔某个“增量”的元素组成），分别进行直接插入排序，然后依次缩减增量再进行排序，待整个序列中的元素基本有序（增量足够小）时，再对全体元素进行一次直接插入排序。

考点　排序算法、希尔排序

#195 408真题

408-2017-10

答案

B

解析

参考排序算法复杂度，归并排序代码比选择插入排序更复杂，前者空间复杂度是O(n), 后者是O(1)。但是前者时间复杂度是O(nlogn), 后者是O(n2)。所以 B 正确。

考点　排序算法、归并排序、插入排序

#196 408真题

408-2017-11

答案

D

解析

插入排序、选择排序、冒泡原本时间复杂度是O(n2)，更换为链式存储后的时间复杂度还是O(n2)。希尔排序和堆排序都利用了顺序存储的随机访问特性，而链式存储不支持这种性质，所以时间复杂度会增加，因此选 D。

考点　排序算法

#197 408真题

408-2018-10

答案

D

解析

参考希尔排序，可以观察到，第一次排序后的数组沿 gap=5 有序，第二次排序后数组沿 gap=3 有序，所以答案选择 D。

考点　排序算法、希尔排序

#198 408真题

408-2018-11

答案

A

解析

参考构造初始堆，该题的堆变化过程如下图所示：

615984761798456971845967184598716 45从结点 5 开始堆化从结点 1 堆化交换5 和 7交换1 和 9从结点 6 堆化交换6 和 8继续向子结点递归交换6 和 8答案选择 A。

考点　堆的概念

#199　408真题

408-2019-7

答案

D

解析

当数据规模较小时可选择是复杂度为O(n2)的简单排序算法，当数据规模较大时应选择复杂度为O(nlog2n)的排序方法，当数据规模大到内存无法放下时需选择外部排序方法，Ⅰ 正确。数据的存储方式主要分为顺序存储和链式存储，有些排序方法（如堆排序）只能用于顺序存储方式，Ⅱ 正确。若对数据稳定性有要求，则不能选择不稳定的排序方法，Ⅲ 显然正确。当数据初始基本有序时，直接插入排序的效率最高，冒泡排序和直接插入排序的时间复杂度都是O(n)，而归并排序的时间复杂度依旧是O(nlog2n)，Ⅳ 正确。所以选 D。

考点　排序算法

#200　408真题

408-2019-10

答案

D

解析

快速排序每趟都将基准元素放在其最终位置，然后以它为基准将序列划分为两个子序列，基准左边都比基准小，右边都比基准大。

排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一“趟”，故快速排序第一趟被枢轴分成的 两个子序列 在第二趟中应该 各放一个枢轴 在最终位置（也即如果第一趟的枢轴在中间，第二趟应该至少有 3 个元素在最终位置），如果枢轴的位置正好在首端或尾端，那第一趟就只被分成了 一个序列，那第二趟只能放 一个枢轴 在最终位置（也即如果枢轴在首尾，第二趟应该至少有 2 个元素在最终位置）

- A 中，第一次快排选的基准值是 72，第二次的基准值是 28。
- B 中，第一次的基准值是 72，第二次的是 2。
- C 中，第一次的基准值是 28，第二次的基准值是 2 和 32。

所以答案选择 D。

考点　排序算法

#201　408真题

408-2020-9

答案

C

解析

堆是一棵完全树，采用一维数组存储，故 I 正确，II 正确。大根堆只要求根结点值大于左右孩子值，并不要求左右孩子值有序，III 错误。堆的定义是递归的，所以其左右子树也是大根堆，所以堆的次大值一定是其左孩子或右孩子，IV 正确。

考点　堆的概念

#202　408真题

408-2020-11

答案

A

解析

考虑较极端的情况，对于有序数组，直接插入排序的比较次数为n−1，简单选择排序的比较次数始终为1+2+⋯+n−1=n(n−1)/2，I 正确。两种排序方法的辅助空间都是O(1)，无差别，II 错误。初始有序时，移动次数均为 0；对于通常情况，直接插入排序每趟插入都需要依次向后挪位，而简单选择排序只需与找到的最小元素交换位置，后者的移动次数少很多，III 错误。

考点　排序算法、插入排序

#203　408真题

408-2021-10

答案

C

解析

LSD 基数排序第一趟根据最低位对数组进行排序，得到的结果为 {151, 301, 372, 892, 93, 43, 485, 946, 146, 267, 327}。

考点　排序算法、基数排序

#204　408真题

408-2021-11

答案

B

解析

参考 构造初始堆 的过程：

669969619615961589815698156498456198456179875614调整调整调整调整

考点　堆的概念

#205　408真题

408-2022-10

答案

A

解析

概念题，归并针对的对象是序列，二路归并就是将两个有序序列合并成一个。

考点　归并排序

#206　408真题

408-2022-11

答案

D

解析

直接插入排序 和 快速排序 的特点如下表所示：

| | 适合初始序列情况 | 适合元素数量 | 空间复杂度 | 稳定性 || --- | --- | --- | --- | --- || 直接插入排序 | 大部分元素有序 | 较少 | O(1) | 稳定 || 快速排序 | 基本无序 | 较多 | O(log2n) | 不稳定 |

I. 大部分元素已有序

✅ 正确。 直接插入排序在序列基本有序的情况下表现非常好，接近 O(n) 的时间复杂度，而快速排序在此场景下仍需要递归处理。因此此项成立。

II. 待排序元素数量很少

✅ 正确。 当元素数量较少时，插入排序的常数因子小、实现简单，实际运行效率往往优于快速排序，这是实际工程中的常见优化策略（如许多库排序在小规模时使用插入排序）。

III. 要求空间复杂度为 O(1)

✅ 正确。 插入排序是原地排序，空间复杂度为 O(1)；而快速排序虽然通常也是原地排序，但某些实现（尤其是递归调用）存在 O(log n) 的栈空间。若对空间要求极严，插入排序更适合。

IV. 要求排序算法是稳定的

✅ 正确。 插入排序是稳定排序，而快速排序是不稳定的（原始版本）。如果应用场景要求排序后相等元素的相对顺序不变，选择插入排序是合理的。

综上，所有说法都正确，正确答案选择 D。

考点　排序算法

#207　408真题

408-2023-10

答案

C

解析

参考排序总结，快速排序在每一轮划分时，通常选择一个枢轴元素将序列分成两部分，并且在交换元素时可能改变相同关键字元素的相对顺序，因此不是稳定的排序算法。堆排序使用堆数据结构进行排序，其中在建堆和调整堆的过程中，元素的交换可能导致相同关键字元素的相对顺序发生改变，因此堆排序也不是稳定的排序算法。希尔排序是基于插入排序的一种改进算法，它通过将待排序的序列划分成若干个较小的子序列进行插入排序，然后逐步缩小子序列的间隔，最终完成整个序列的排序。由于希尔排序是通过跳跃式的插入排序进行排序的，相同关键字的元素可能会跨越较大的间隔进行比较和交换。这种跨越较大间隔的比较和交换可能导致相同关键字的元素的相对顺序发生改变，因此希尔排序是不稳定的排序算法。因此不稳定的排序算法有希尔排序、快速排序、堆排序。本题答案选 C。

考点　排序算法

#208　408真题

408-2023-11

答案

D

解析

在快速排序中，划分过程通常选择一个枢轴元素来将待排序序列划分为两个子序列。因为是升序排序，所以枢纽元素的前半个子序列的值需要都小于枢轴值，后半个子序列的值需要都大于枢轴值。分别从每个选项来看，A 选项的枢轴值为 11，前半个子序列的值只有68，大于枢轴值 11，不符合。B 选项的枢轴值为 70，前半个子序列都小于 70，但后半个子序列存在 23 和 48 小于 70，不符合。C 选项的枢轴值为 80，前半个子序列的值都小于 80，但是后半个子序列存在 48 小于 80，不符合。D 选项的枢轴值为 81，81 前面的元素都小于 81，81 后面的元素都大于 81，符合。因此本题的正确选项为 D。

考点　排序算法

#209　408真题

408-2024-8

答案

A

解析

本题考察快排的原理，快排第一趟找到枢轴之后，枢轴的左边的元素即 P 块都是小于枢轴的，枢轴的右边的元素即 Q 块是大于枢轴的，所以 P 和 Q 块间有序，然而块内未必是有序的，所以还需要继续对 P 和 Q 两块进行递归快排。

考点　快速排序

#210　408真题

408-2024-9

答案

B

解析

本题考察 堆的删除操作，先根据初始序列建堆，然后依次删除堆项元素，过程如下图：

2822201981215552220198121522192058121515192058122019155812堆的初始形态删除 28调整删除 22调整

考点　堆的概念

#211　408真题

408-2024-10

答案

C

解析

本题考察 多路归并 的原理，二路归并排序将相邻的有序子序列两两合并，得到合并后有序子序列。重复此过程，直到所有子序列合并为一个有序序列。

第一轮归并：将升序序列两两分组，(3,5) 和 (7,9) 分为一组，(6) 分为一组。(3,5) 和 (7,9) 的归并过程如下：

| 输入序列 | 比较首个元素 | 输出元素 | 输出序列 | 剩余输入序列 || --- | --- | --- | --- | --- || (3,5) 和 (7,9) | 3 < 7 | 3 | 3 | (5) 和 (7,9) || (5) 和 (7,9) | 5 < 7 | 5 | 3,5 | (7,9) || (7,9) | | 7 | 3,5,7 | (9) || (9) | | 9 | 3,5,7,9 | |

第二轮归并：将升序序列两两分组，(3,5,7,9) 和 (6) 分为一组。(3,5,7,9) 和 (6) 的归并过程如下：

| 输入序列 | 比较首个元素 | 输出元素 | 输出序列 | 剩余输入序列 || --- | --- | --- | --- | --- || (3,5,7,9) 和 (6) | 3 < 6 | 3 | 3 | (5,7,9) 和 (6) || (5,7,9) 和 (6) | 5 < 6 | 5 | 3,5 | (7,9) 和 (6) || (7,9) 和 (6) | 7 > 6 | 6 | 3,5,6 | (7,9) || (7,9) | | 7 | 3,5,6,7 | (9) || (9) | | 9 | 3,5,6,7,9 | |

到此，二路归并排序执行完毕。

上述过程中第一轮归并关键字之间比较 2 次，第二轮归并关键字之间比较 3 次，因此，关键字之间的总比较次数为 2 + 3 = 5。

本题选 C。

考点　归并排序

#212　408真题

408-2025-10

答案

D

解析

排序算法的元素移动次数参考此节，其中简单选择排序迭代 n 轮，每次确定一个元素的最终位置，元素移动次数平均为O(n)。

考点　排序算法

#213　408真题

408-2025-11

答案

A

解析

第一次排序后，下标相差 3 的子序列有序。第二次排序后，下标相差 2 的子序列有序。由此判断是希尔排序。

考点　排序算法

#214　408真题

408-2026-9

答案

B

解析

【解析】 直接插入排序的比较次数取决于序列的初始有序程度。对于每个序列，从第二个元素开始，将其与前面已排序的元素从后往前比较，直到找到正确位置，记录比较次数。

- 选项 A：序列 30, 27, 56, 41, 80, 95, 69 的总比较次数为1+1+2+1+1+3=9次。
- 选项 B：序列 31, 43, 26, 55, 63, 99, 77 的总比较次数为1+2+1+1+1+2=8次。
- 选项 C：序列 61, 84, 51, 23, 34, 91, 40 的总比较次数为1+2+3+4+1+5=16次。

- 选项 D：序列 93, 32, 48, 81, 50, 21, 72 的总比较次数为1+2+2+3+5+3=16次。

比较次数最少的是选项 B，共 8 次。

考点　插入排序

#215 408真题

408-2026-10

答案

A

解析

【解析】排序规则要求先按 C1 成绩升序，再按总分升序，这属于多键排序问题。基数排序是一种稳定的排序算法，特别适合多键排序，因为它可以对每个键位进行独立排序，且稳定性保证了当主键（C1）相同时，次键（总分）的顺序得以保持。具体实现时，可以先按总分（低优先级键）进行稳定排序，再按 C1（高优先级键）进行稳定排序，从而满足规则。其他算法中，快速排序、希尔排序和选择排序都不是稳定的，虽然可以通过自定义比较函数在一次排序中处理多键，但稳定性和效率不如基数排序。此外，学生成绩通常为整数，基数排序对整数排序效率较高。因此，基数排序是最适合的算法。

考点　排序算法

填空题 29 题

第二章　线性表 1 题

#216 802真题2026

PDF P3 · 真题-2026-填空12

答案

O(n)

解析

需从头查找定位第 i 个结点，平均 O(n)。

考点　链表插入、时间复杂度

第三章　栈队列 7 题

#217 802真题2024

PDF P4 · 真题-2024-填空1

答案

(r − f + n) % n

解析

f 指向队头元素的前一个位置、r 指向队尾元素。队长 = (rear − front + maxsize) % maxsize，此处即 (r − f + n) % n。

考点　循环队列、队长公式

#218 802真题2024

PDF P4 · 真题-2024-填空2

答案

ab+acd+e/f-*-g+

解析

用运算符栈逐个扫描：操作数直接输出；遇运算符弹出栈中优先级不低于它的运算符后再入栈；遇 '(' 入栈、遇 ')' 弹至 '('。逐字符转换后得后缀 ab+acd+e/f-*-g+，与答案PDF一致(已核对无误)。

考点　中缀转后缀、运算符栈

#219 802真题2025

PDF P10 · 真题-2025-填空2

答案

答案PDF给出：9。

解析

按题面严格分析：要使 4 第一个出栈，必须先把 1,1,2,3,4 全部依次入栈（期间不能有任何出栈，否则首元素不是 4），随后弹出 4；此时栈内自底向上为 1,1,2,3，且已无入栈操作，只能按 LIFO 依次弹出

3,2,1,1。故以 4 开头的出栈序列只有唯一一种：4,3,2,1,1，正确答案应为 1。

【勘误·以修正版为准】 【答案PDF有误】答案PDF给出 9，其解析误用 Catalan 数：把'4 先出栈后剩下的 1,1,2,3'当作仍有 push/pop 交错自由的子问题（C(4)=14 再去重 5 得 9）。但 4 最后入栈、最先出栈时，1,1,2,3 必已全部在栈中，出栈顺序被 LIFO 完全锁定，正确结论应为 1 个（4,3,2,1,1）。答案PDF实际算的是'1,1,2,3 四元素全部不同出栈序列去重数'这一另一问题。

考点　栈、出栈序列、LIFO

#220　802真题2025

PDF P10　·　真题-2025-填空5

答案

-19。

解析

用栈求值：1→栈[1]；2→[1,2]；4→[1,2,4]；2→[1,2,4,2]；/ 弹 2、4 算 4/2=2 入栈→[1,2,2]；+ 弹 2、2 算 2+2=4→[1,4]；5→[1,4,5]；* 弹 5、4 算 4×5=20→[1,20]；- 弹 20、1 算 1-20=-19。结果 -19。

考点　后缀表达式、逆波兰式、求值

#221　802真题2026

PDF P3　·　真题-2026-填空6

答案

9

解析

还原为中缀 a*(b+c)-d = 2*(3+4)-5 = 9。

考点　后缀表达式求值

#222　802真题2026

PDF P3　·　真题-2026-填空7

答案

front=4，rear=2

解析

删除一个元素 front=(3+1)%6=4；加入两个元素 rear=(0+2)%6=2。

【勘误·以修正版为准】 答案 PDF 末尾写'答案：3'与其自身解析(front=4,rear=2)矛盾，疑为笔误；以解析过程为准，取 front=4、rear=2。

考点　循环队列、front/rear指针

#223　802真题2026

PDF P3　·　真题-2026-填空9

答案

n-i+1

解析

T1=n 说明所有元素先全部入栈再开始出栈，出栈即从 n 递减，第 i 个出栈为 n-i+1。

备注：回忆版题面文字 OCR 有断裂(题号与题干错位)，已按答案 PDF 的解读补全为完整题意。

考点　出栈序列、栈的LIFO

第四章　串 2 题

#224 802真题2024

PDF P4 · 真题-2024-填空3

答案

next[1..12] = 0, 1, 1, 1, 2, 2, 3, 1, 2, 3, 4, 5

解析

按严蔚敏 next 定义(next[1]=0，next[j]=最长相等前后缀长度+1)逐位递推。例如 j=7 时前 6 位 'abcaab' 的最长相等前后缀为 'ab'(长2)，故 next[7]=3。整表已逐位核对，与答案PDF一致。

考点　KMP、next数组

#225 802真题2025

PDF P10 · 真题-2025-填空8

答案

next=0111223（王道约定：next[1]=0，共 7 位对应 7 个字符）。 逐位：a→0, b→1, c→1, a→1, a→2, b→2, a→3。

解析

按王道约定 next[j] 基于前缀子串 p[1..j-1] 的最长相等前后缀长度 k，next[j]=k+1（next[1]=0,next[2]=1）： next[1]=0；next[2]=1； next[3]：'ab' 无相等前后缀→1；next[4]：'abc'→1； next[5]：'abca' 最长相等前后缀 'a'(长1)→2； next[6]：'abcaa' 最长 'a'(长1)→2； next[7]：'abcaab' 最长 'ab'(长2)→3。 故 next=0,1,1,1,2,2,3。

【勘误•以修正版为准】 【答案PDF有误】答案PDF写 '011123'（仅6位），少一位，疑为 OCR 漏字或笔误。7 个字符应有 7 位 next，正确为 0111223（王道 next[1]=0 约定）。若采用 nextval 或 0 基约定数值会不同，但按题面'next 数组'与王道约定应为 0111223。

考点　KMP、next数组

第五章　数组广义表 5 题

#226 802真题2024

PDF P4 · 真题-2024-填空6

答案

709

解析

行优先：LOC(A[i][j]) = LOC(A[0][0]) + (i×列数 + j)×元素大小 = 644 + (4×15 + 5)×1 = 644 + 65 = 709。

考点　二维数组地址计算、行优先

#227　802真题2025

PDF P10　·　真题-2025-填空3

答案

1376。

解析

行优先四维数组 A[n1][n2][n3][n4]，元素占 M 字节：LOC(A[i][j][k][l])=LOC(A[0][0][0][0])+(i·n2n3n4+j·n3n4+k·n4+l)·M。代入 n2=3,n3=5,n4=8,M=4：100+(2×3×5×8+1×5×8+4×8+7)×4=100+(240+40+32+7)×4=100+319×4=1376。

备注：OCR错位已重组：原题面维度 OCR 为散乱的'8 5 3 9'且下标'A2147'，已按答案计算式 2×3×5×8+1×5×8+4×8+7 反推为行优先数组 A[9][3][5][8] 求 A[2][1][4][7]。

考点　多维数组、行优先、地址计算

#228　802真题2025

PDF P10　·　真题-2025-填空7

答案

(c,d)。

解析

广义表 LS=((a,b),(c,d))。getTail(LS) 去掉表头 (a,b)，余下部分仍为表 ((c,d))；再 getHead(((c,d))) 取其表头 = (c,d)。故结果为 (c,d)（注意是一个含两个原子的子表，不带最外层括号）。

考点　广义表、取表头getHead、取表尾getTail

#229　802真题2026

PDF P3　·　真题-2026-填空8

答案

300

解析

设每行 N 个元素：A[3][3]=100+(3N+3)=220 → N=39；A[5][5]=100+(5×39+5)=100+200=300。

考点　二维数组地址计算、行优先

#230　802真题2026

PDF P3　·　真题-2026-填空14

备注：【题面严重缺失】回忆版仅有'是一个广义表的运算取头取尾'一句，无具体广义表与所求。答案 PDF 亦注'题目不全，通常考察 Head() 取表头、Tail() 取表尾'。保留占位，待主讲补全具体题面。

考点　广义表、取表头表尾

第六章　树 8 题

#231 802真题2024

PDF P4 · 真题-2024-填空4

答案

624

解析

哈夫曼树只有度为 0 和度为 2 的结点，满足 N0 = N2 + 1，故总结点 N = N0 + N2 = 2·N0 − 1。由 2·N0 − 1 = 1247 得 N0 = 624。

考点　哈夫曼树、结点度关系

#232 802真题2024

PDF P4 · 真题-2024-填空7

答案

1896

解析

树转二叉树(孩子兄弟法)中，结点的右孩子=其下一个兄弟，故'右子树为空'=该结点无右兄弟=是其父的最右孩子，或本身是树根。分支结点数=总结点−叶=2011−116=1895，每个分支结点恰有一个最右孩子(无右兄弟)，共 1895 个；再加树根(无兄弟)1 个，合计 1896。

考点　树转二叉树、孩子兄弟表示、右子树为空

#233 802真题2024

PDF P4 · 真题-2024-填空8

答案

M1 − 1

解析

森林转二叉树：二叉树的根=第一棵树的根；根的左子树=第一棵树中根的子树森林所转的二叉树；根的右子树=其余各树(M2、M3)所转的二叉树。故根结点左子树上的结点数=第一棵树除根外的全部结点=M1−1。

备注：OCR 中题号前混入'3E'为噪声，已剔除。

考点　森林转二叉树、左子树结点数

#234 802真题2024

PDF P4 · 真题-2024-填空9

答案

111

解析

要使结点最多，树高取 7 且第 6 层尽量满。第 6 层最多 2^5=32 个结点，其中 8 个为叶(必是最右侧 8 个)，则左侧 32−8=24 个结点在第 7 层各有 2 个孩子，第 7 层有 24×2=48 个。前六层满=2^6−1=63，共 63+48=111；亦即 (2^7−1)−8×2=127−16=111。

考点　完全二叉树、叶结点、最多结点数

#235 802真题2025

PDF P10 · 真题-2025-填空4

答案

100。

解析

哈夫曼树（严格二叉树/正则二叉树）只有度为 0 和度为 2 的结点，满足 n0=n2+1，总结点数 n=n0+n2=2n0-1。由 2n0-1=199 得 n0=100。

考点　哈夫曼树、结点数性质

#236 802真题2025

PDF P10 · 真题-2025-填空6

答案

10。

解析

一棵有 n 个结点的树恰有 n-1 条边。森林含 k 棵树、总结点 V、总边 E，则 E=V-k。代入 V=25,E=15 得 k=V-E=25-15=10。

考点　森林、结点与边关系

#237 802真题2025

PDF P10 · 真题-2025-填空9

答案

82。

解析

设叶结点(度0) n0 个。总结点数 n=n0+n1+n2+n3+n4=n0+10+1+10+20=n0+41。总边数=各结点度数之和=1×10+2×1+3×10+4×20=10+2+30+80=122。树中边数=结点数-1，故 122=(n0+41)-1=n0+40，得 n0=82。（等价公式 n0=1+n2+2n3+3n4=1+1+20+60=82。）

考点　树的度、叶结点数、边数=结点数-1

#238 802真题2026

PDF P3 · 真题-2026-填空11

答案

n+1

解析

n 个结点共 2n 个指针域，其中 n-1 个用于连接(边数)，空指针 = 2n-(n-1) = n+1。

考点　二叉树、空指针数、二叉链表

第七章　图 1 题

#239 802真题2026

PDF P3 · 真题-2026-填空10

答案

n

解析

强连通有向图至少需要一个包含全部 n 个顶点的有向环，恰好 n 条边。

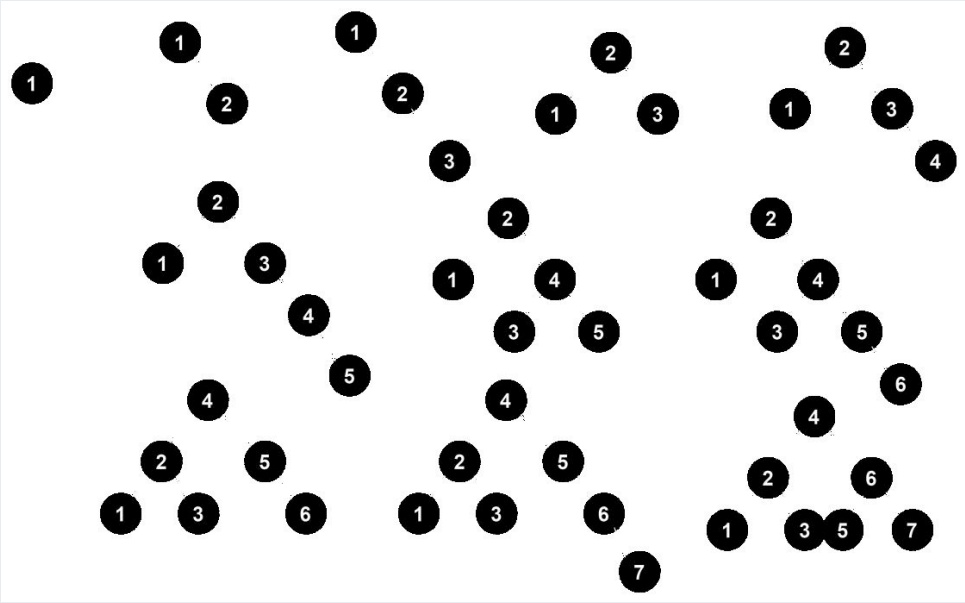
【勘误•以修正版为准】 【答案 PDF 有误】其写'需要 n+1 条边构成环'。强连通有向图最少边数为 n(n 个顶点构成一个有向环)，非 n+1；已按正确结论入库。

考点 强连通图、最少边数

第八章 查找 2 题

#240 802真题2024

PDF P4 · 真题-2024-填空5



图：z2425-p16-0.png

答案

7

解析

依次插入并旋转平衡后得到满二叉树：根 4，左子树 2(1,3)，右子树 6(5,7)。7 个结点全部左右子树等高，平衡因子均为 0，故为 7。(已逐次插入+旋转核对，与答案PDF一致)

备注：构造过程图见答案PDF P16(z2425-p16-0.png)。

考点 平衡二叉树AVL、顺序插入、平衡因子

#241 802真题2026

PDF P3 · 真题-2026-填空13

答案

7

解析

最多比较次数 = $\lfloor \log_2 125 \rfloor + 1 = 6 + 1 = 7$(判定树高度)。

考点　折半查找、最大比较次数

第九章　排序 3 题

#242 802真题2025

PDF P9 · 真题-2025-填空1

答案

O(n^2)。

解析

递归版冒泡排序：F(a,i,n) 做一趟从 j=i 到 n-2 的相邻比较交换（约 n-1-i 次），再递归 F(a,i+1,n)。总比较次数 Σ(i=0..n-2)(n-1-i)=n(n-1)/2，即 O(n^2)。输入 {5,4,3,2,1} 为逆序最坏情况，但代码无提前终止，任何输入均为 O(n^2)。

备注：题干代码跨 P9-P10 两页，已按逻辑拼接完整。

考点　递归、冒泡排序、时间复杂度

#243 802真题2025

PDF P10 · 真题-2025-填空10

解析

小根堆插入：新元素放到堆末尾，再与其父结点比较，若小于父结点则交换（上浮），直到不小于父结点或到根。具体结果取决于原堆序列。

备注：【图缺失】题干'如图所示'的原始小根堆图未在题面(P10)给出，也无提取到的图片；答案PDF(P31-32)仅文字'根据堆的调整规则调整即可得到'，未给出插入3后的堆序列。缺原堆数据无法确定唯一答案，仅作真题登记。

考点　小根堆、插入调整、上浮

#244 802真题2026

PDF P3 · 真题-2026-填空15

答案

$\lfloor \log_2 n \rfloor$（最坏情况，堆的深度）

解析

新元素插入堆底后向上调整，最坏情况从叶子比较到根，比较次数为树高 $\lfloor \log_2 n \rfloor$。

备注：回忆版题干为'如图所示'但 26 回忆版未给出该图，按大根堆插入向上调整的通用结论入库。

考点　大根堆、插入调整、比较次数

程序阅读题 14 题

第一章　绪论 3 题

#245 802真题2024

PDF P1 · 真题-2024-阅读1

答案

时间复杂度为 $O(\log_2(\log_2 n))$，即 O(log log n)。

解析

循环变量 x 从 2 开始每次平方：x 依次取 2, 2^2, 2^4, 2^8, …, 2^(2^k)。终止条件 $x \geq n/2$，即 $2^{(2^k)} \geq n/2$。两边取对数得 $2^k \geq \log_2(n/2)=\log_2 n-1$，再取对数得 $k \geq \log_2(\log_2 n-1)\approx\log_2\log_2 n$。故迭代次数(即 count)为 O(log log n)。

考点　时间复杂度分析、对数级迭代

#246 802真题2025

PDF P7 · 真题-2025-阅读1

答案

时间复杂度为 O(2^n)。

解析

这是汉诺塔递归。设 T(n) 为移动 n 个盘子的操作数，则 T(n)=2T(n-1)+1，T(1)=1，解得 T(n)=2^n-1，故为 O(2^n)。递归树高度为 n，第 i 层约有 2^(i-1) 个结点，总操作数指数级。

备注：OCR错位已重组：原 OCR 在 move() 中混入一行无定义的 printf("%s\n", buffer); 及乱序行号，已按汉诺塔逻辑删除并还原。

考点　汉诺塔、递归、时间复杂度

#247 802真题2026

PDF P2 · 真题-2026-阅读4

答案

版本一输出：`23 16 16 34 34 29 29`，时间复杂度 O(n)； 版本二无打印、返回值为 16，时间复杂度 O(n)。

解析

版本一递归到 n=1 返回后，自底向上打印每对相邻元素的较小者；版本二递归求前缀最小值。两者均线性递归，O(n)。

> **【勘误•以修正版为准】** 回忆版作者说明：'本题作者似乎寄了，不确定是以下哪个函数或其他'——两个版本并存，题目本身存疑，讲解时以版本一为准并说明歧义。

考点　递归、时间复杂度、程序输出、函数阅读

第二章　线性表 5 题

#248 802真题2024

PDF P1 · 真题-2024-阅读2

答案

四个空自上而下依次填：③ ① ④ ②。 即：空1=③(边界检查)、空2=①(未找到 p 且未到表尾则继续)、空3=④(prev 的后继正是 p)、空4=②(prev->next=s)。 时间复杂度 O(n)(n 为链表长度，需从头遍历找到 p 的前驱)。

解析

插入到 p 之前须先定位 p 的前驱 prev。先做非法指针检查(③)；while 用 prev->next!=p && prev->next!=NULL 向后扫描(①)；找到后用 prev->next==p 确认命中(④)，再 s->next=p、prev->next=s 完成插入(②)。

备注：无贴切模板(前驱遍历插入不属于给定模板集)，template 留空。

考点　单链表插入、前驱查找、代码片段排序、时间复杂度

#249 802真题2024

PDF P2 · 真题-2024-阅读3

答案

① `La != NULL && Lb != NULL` ② `La->data <= Lb->data` ③ `p->next = Lc` ④ `Lc = p` 时间复杂度 O(n+m)。

解析

每次取两表当前较小结点，用头插法(p->next=Lc; Lc=p)插入 Lc 头部；由于两表非递减、每次取较小者且头插，最终 Lc 呈非递增。任一表遍历完后，把剩余结点同样头插到 Lc 头部。每个结点仅访问一次，故 O(n+m)。

备注：题面中 hc 声明后未使用，属冗余；实际以 Lc 直接头插构建。

考点　头插法　|　链表合并、头插法逆序、非递减转非递增、时间复杂度

#250 802真题2025

PDF P7 · 真题-2025-阅读2

答案

答案PDF给出：① p->prior = q->prior; ② q->next = p->next;

【正确重组代码】（须在 p->next=q 之前保存 p 的原后继 y）：

```
void SwapWithPrevious(DLinkNode* p) {
    DLinkNode* q = p->prior;
    DLinkNode* y = p->next;   // 先保存 p 的原后继
    p->prior = q->prior;      // ①
    p->next = q;
    q->prior = p;
    q->next = y;              // ② 用原后继 y，不能用已被覆盖的 p->next
    if (p->prior != NULL) p->prior->next = p;
    if (q->next != NULL) q->next->prior = q;
}
```

解析

交换 p 与其前驱 q：需重接 6 个指针——p、q 各自的 prior/next，以及 p 原前驱 X(q->prior) 与 p 原后继 Y(p->next) 的回指。关键：p->next=q 会覆盖 Y，故必须先把 Y=p->next 存入临时变量，才能让 q->next=Y。

> **【勘误•以修正版为准】** 【答案PDF有误】答案PDF的② q->next = p->next 在骨架中位于 p->next=q 之后执行，此时 p->next 已被改为 q，会导致 q->next=q（自环），并使随后的 q->next->prior=q 把 q->prior 破坏为 q。正确做法是先保存 p 的原后继再赋给 q->next（见重组代码）。此错也可能源于回忆版骨架行序 OCR 错位。

考点　双向链表、结点交换、指针操作

#251　802真题2025

PDF P9　·　真题-2025-阅读4

答案

① p = L->next; ② min != p; ③ p = p->next;

解析

带头结点，故 p 从第一个有效结点 L->next 开始。每趟在 p 及其后用 min 记录最小值结点，若 min!=p 则交换两结点的 data（交换值而非结点），最后 p 后移进入下一趟。

考点　带头结点单链表、简单选择排序

#252　802真题2026

PDF P1　·　真题-2026-阅读3

答案

空1：`pa && pb`　空2：`pc->next = pa;`（相等时把 pa 尾插到结果链，随后 pc=pa）　空3：`pc->next = NULL;`（收尾）

解析

归并式求交集：双指针同步后移，相等则保留 pa 结点接到 pc 之后（复用 A 空间）、删除 pb 结点；不等则删除较小者。要求空间复杂度 O(1)（利用原结点）。

备注：回忆版代码有跨页断裂，已按逻辑补全；空2 依答案 PDF 为 pc->next=pa。

考点　双指针/快慢指针　|　链表、求交集、双指针、尾插、空间复用

第三章　栈队列　2 题

#253　802真题2025

PDF P8　·　真题-2025-阅读3

答案

① p->next = Q->rear->next; ② Q->rear = NULL;

解析

①入队：新结点 p 的 next 先指向队头(=Q->rear->next)，再让原尾结点指向 p，最后 rear 更新为 p，保持循环。②出队：当 Q->rear==p 说明队列只剩一个结点，出队后置 Q->rear=NULL 使队列为空。

考点　循环单链表、链式队列、入队出队

#254 802真题2026

真题-2026-阅读2

答案

空1：`S.top[0] + 1 == S.top[1]`（两栈顶相邻即满） 空2：`++S.top[0]` 空3：`--S.top[1]` 空4：`S.top[0]--` 空5：`S.top[1]++`

解析

一个数组实现两个栈，栈底分别在数组两端向中间生长。栈0 从左往右（先加指针再存值，出栈先取值再减指针），栈1 从右往左（先减指针再存值，出栈先取值再加指针）。

备注：回忆版注明'红色部分并未给出'；OCR 把空位与 S.data[__]=x 打乱，已按共享栈逻辑与答案重组。

考点　共享栈、双栈、进栈出栈、栈满判定

第六章　树 2 题

#255 802真题2024

PDF P3 · 真题-2024-阅读5

答案

该代码递归计算并返回二叉树中叶结点(度为 0 的结点)的数量。

解析

空树返回 0；当前结点左右孩子皆空即为叶结点返回 1；否则返回左右子树叶结点数之和。属典型二叉树递归三件套(终止条件+单层逻辑+返回值合并)。

考点　递归三件套 | 二叉树递归、叶结点计数

#256 802真题2026

PDF P3 · 真题-2026-阅读5

答案

空1：`IsBalanced(T->lchild, deepleft) && IsBalanced(T->rchild, deepright)` 空2：`abs(dif) <= 1`（或 `dif >= -1 && dif <= 1`）

解析

后序遍历判断平衡性：空树深度0返回true；递归检查左右子树是否都平衡并取得各自深度，再判左右深度差绝对值是否≤1，平衡则当前深度为较大者+1。

考点　递归三件套 | 平衡二叉树、AVL、后序遍历、递归求深度

第七章　图 2 题

#257 802真题2024

PDF P3 · 真题-2024-阅读4

答案

① `Push(S, i)` ② `!IsEmpty(S)` ③ `Pop(S, i)` ④ `p != NULL` 时间复杂度 O(|V|+|E|)。

解析

先把所有入度为 0 的顶点压栈；栈非空则弹栈顶 i 输出，遍历 i 的邻接边把后继入度减 1，减为 0 者压栈；最终输出顶点数 < 顶点总数说明有回路，拓扑排序失败。每个顶点、每条边各处理一次，故 O(|V|+|E|)。

考点　入度表法　|　拓扑排序、入度表、栈、时间复杂度

#258　802真题2026

真题-2026-阅读1

答案

空1：`p = G.vertices[v].firstarc`（取 v 的第一条依附边）　空2：`DFSTree(G, p->adjvex)`（递归进入未访问的邻接点）

解析

考察图的 DFS 遍历及生成树构建。访问顶点 v 后先取其第一条依附边，再对每个未访问的邻接点递归，递归边即生成树的树边。

备注：回忆版此处代码顺序 OCR 有错位（printf 与空2 位置颠倒），已按 DFS 逻辑与答案重组：空2 在 if(!visited) 块内。

考点　DFS回溯　|　DFS、深度优先生成树、邻接表遍历

综合应用题 43 题

第二章　线性表 3 题

#270 408真题

408-2015-41

答案

1）算法的基本设计思想

算法的核心思想是用空间换时间。使用辅助数组记录链表中已出现的数值，从而只需对链表进行一趟扫描。

因为|data|≤n，故辅助数组q的大小为n+1，各元素的初值均为 0。依次扫描链表中的各结点，同时检查q[|data|]的值，如果为0，则保留该结点，并令q[data]=1；否则，将该结点从链表中删除。

2）使用 C 语言描述的单链表结点的数据结构定义：

3）算法实现

```
void RemoveElements(ListNode *head, int n) {
// 数组充当哈希表
int hash[n+1];
for (int i = 0; i <= n; i++) {
// 0 表示没有命中
hash[i] = 0;
}
// 当遍历的结点
ListNode *q = head->link;
// 先前的结点
ListNode *p = head;
while (q != NULL) {
// 绝对值已经出现过
if (hash[abs(q->data)] == 1) {
// 删除该节点
p->next = q->next;
q = p->next;
} else {
// 设置哈希表
hash[abs(q->data)] = 1;
q = q->next;
p = p->next;
}
}
}
```

【评分说明】若考生设计的算法满足题目的功能要求且正确，则酌情给分。

4）参考答案所给算法的时间复杂度为 O(m)，空间复杂度为 O(n)。

【评分说明】若考生所估计的时间复杂度和空间复杂度与考生实现的算法一致，可给分。

考点　链表

#271 408真题

答案

1）算法的基本设计思想：先观察 L(a1,a2,a3,⋯,an−2,an−1,an) 和 L’ (a1,an,a2,an−1,a3,an−2,⋯)，发现 L’ 是由 L 摘取第一个元素，再摘取倒数第一个元素⋯依次合并而成的。为了方便链表后半段取元素，需要先将 L 后半段原地逆置［题目要求空间复杂度为 助栈］，否则每取最后一个结点都需要遍历一次链表。

①先找出链表 L 的中间结点，为此设置两个指针 p 和 q，指针 p 每次走一步，指针 q 每次走两步，当指针 q 到达链尾时，指针 p 正好在链表的中间结点；

②然后将 L 的后半段结点原地逆置。

③从单链表前后两段中依次各取一个结点，按要求重排。

2）算法实现如下：

```
// 找到链表的中间结点
NODE *findMiddleNode(NODE *head) {
NODE *slow = head;
NODE *fast = head;
while (fast != NULL) {
fast = fast->next;
if (fast != NULL) {
fast = fast->next;
}
slow = slow->next;
}
return slow;
}

// 反转链表
NODE *reverse(NODE *start) {
NODE *p = start;
NODE *q = start->next;
while (q != NULL) {
NODE *tmp = q->next;
q->next = p;
p = q;
q = tmp;
}
return p;
}

// 1. 找到中位结点
// 2. 翻转后一半链表
// 3. 遍历两个链表，依次穿插所有结点
void solve(HEAD *head) {
if (head->next == NULL) {
return;
}
NODE *middle = findMiddleNode(head);
NODE *list2 = reverse(middle);
NODE *list1 = head->next;
// 穿插操作
NODE *p = list1;
NODE *q = list2;
// 退出循环的条件
```

```
// 链表长度为奇数：p == q
// 链表长度为偶数：p->next == q
// while (!(p == q || p->next == q))
while (p != q && p->next != q) {
NODE *tmp1 = p->next;
NODE *tmp2 = q->next;
p->next = q;
q->next = tmp1;
p = tmp1;
q = tmp2;
}
}
```

3）第 1 步找中间结点的时间复杂度为 O(n)，第 2 步逆置的时间复杂度为 O(n)，第 3 步合并链表的时间复杂度为 O(n)，所以该算法的时间复杂度为 O(n)。

考点　链表

#272　408真题

408-2020-41

答案

分析，由 D=|a−b|+|b−c|+|c−a|≥0得：

① 当 a=b=c时，距离最小。

② 其余情况。不失一般性，假设观察下面的数轴：

L1L2L3abcL1=|a−b|，L2=|b−c|，L3=|c−a|，D=|a−b|+|b−c|+|c−a|=L1+L2+L3=2L3

由D的表达式可知，事实上决定D大小的关键是a和c之间的距离，于是问题就可以简化

为每次固定c找一个a使得L3=|c−a|最小。

1）算法的基本设计思想

- 使用Dmin记录所有已处理过的三元组的最小距离，初值为一个足够大的整数。
- 集合S1、S2和S3，分别保存在数组A、B、C中。数组的下标变量i=j=k=0，当i<S1、j<S2且k<S3时（|S|表示集合S中的元素个数），循环执行以下过程：计算（A[i]，B[j]，C[j]）的距离D；（计算D）若D<Dmin，则Dmin=D；（更新D)将A[i]，B[i]，C[j]中的最小值的下标+1；（对照分析：最小值为a，最大值为c，这里c不变而更新a，试图寻找更小距离D）
- 计算（A[i]，B[j]，C[j]）的距离D；（计算D）
- 若D<Dmin，则Dmin=D；（更新D)
- 将A[i]，B[i]，C[j]中的最小值的下标+1；（对照分析：最小值为a，最大值为c，这里c不变而更新a，试图寻找更小距离D）
- 输出Dmin，结束。

2）算法实现

```
void solve(int S1[], int n1, int S2[], int n2, int S3[], int n3) {
int i = 0, j = 0, k = 0;
int res = INT32_MAX;
while (i < n1 && j < n2 && k < n3) {
int a = S1[i], b = S2[j], c = S3[k];
int D = abs(a-b) + abs(b-c) + abs(a-c);
res = min(res, D);
int v = min3(a, b, c);
```

```
if (v == a) {
i++;
} else if (v == b) {
j++;
} else {
k++;
}
}
return res;
}
```

3）设 n=|S1|+|S2|+|S3|，时间复杂度为O(n)，空间复杂度为O(1)。

考点　顺序表

第三章　栈队列 3 题

#278 408真题

408-2017-41

答案

1）算法的基本设计思想

表达式树的中序序列加上必要的括号即为等价的中缀表达式。可以基于二叉树的中序遍历策略得到所需的表达式。（3 分）

表达式树中分支结点所对应的子表达式的计算次序，由该分支结点所处的位置决定。为得到正确的中缀表达式，需要在生成遍历序列的同时，在适当位置增加必要的括号。显然，表达式的最外层（对应根结点）及操作数（对应叶结点）不需要添加括号。（2 分）

2）算法实现（10 分）

```
void preorder(node *root, int depth) {
if (!root) {
return;
}
bool isRoot = (depth == 1);
bool isLeaf = (!root->left && !root->right);
if (!isRoot && !isLeaf) {
printf("(");
}
preorder(root->left, depth+1);
printf("%s", root->data);
preorder(root->right, depth+1);
if (!isRoot && !isLeaf) {
printf(")");
}
}

void solve(node *root) {
preorder(root, 1);
}
```

将二叉树的中序遍历递归算法稍加改造即可得本题答案。除根结点和叶结点外，遍历到其他结点时在遍历其左子树之前加上左括号，在遍历完右子树后加上右括号。

【评分说明】①若考生设计的算法满足题目的功能要求，则（1)、(2) 根据所实现算法的策略及输出结果给分，评分标准见下表。

| 分数 | 备注 || --- | --- || 15 | 采用中序遍历算法且正确，括号嵌套正确，层数适当 || 14 | 采用中序遍历算法且正确，括号嵌套正确，但括号嵌套层数过多。例如，表达式最外层加上括号，或操作数加括号，如(a) || 11 | 采用中序遍历算法，但括号嵌套层数不完全正确。例如，左右括号数量不匹配 || 9 | 采用中序遍历算法，但没有考虑括号 || ≤7 | 其他 |

②若考生采用其他方法得到正确结果，可参照①的评分标准给分。

③如果程序中使用了求结点深度等辅助函数，但没有给出相应的实现过程，只要考生进 行了必要的说明，可不扣分。

④若在算法的基本设计思想描述中因文字表达没有清晰反映出算法思路，但在算法实现 中能够表达出算法思想且正确的，则可参照①的标准给分。

⑤若算法的基本设计思想描述或算法实现中部分正确，可参照①中各种情况的相应给分标准酌情给分。

考点　二叉树遍历

#279 408真题

408-2019-42

答案

1）顺序存储无法满足要求②的队列占用空间随着入队操作而增加。根据要求来分析：要求①容易满足；链式存储方便开辟新空间，要求②容易满足；对于要求③，出队后的结点并不真正释放，用队头指针指向新的队头结点，新元素入队时，有空余结点则无须开辟新空间，赋值到队尾后的第一个空结点即可，然后用队尾指针指向新的队尾结点，这就需要设计成一个首尾相接的循环单链表，类似于循环队列的思想。设置队头、队尾指针后，链式队列的入队操作和出队操作的时间复杂度均为 O(1)，要求④可以满足。因此，采用链式存储结构（两段式单向循环链表），队头指针为 front，队尾指针为 rear。

2）该循环链式队列的实现，可以参考循环队列，不同之处在于循环链式队列可以方便增加空间，出队的结点可以循环利用，入队时空间不够也可以动态增加。同样，循环链式队列也要区分队满和队空的情况，这里参考循环队列牺牲一个单元来判断。初始时，创建只有一个空闲结点的循环单链表，头指针 front 和尾指针 rear 均指向空闲结点，如下图所示。

frontrear队空的判定条件：front == rear。

队满的判定条件：front == rear->next。

3）插入第一个元素后的状态如下图所示。

frontrear4）操作的基本过程：

入队操作

```
if (front == rear->next)
则在 rear 后面插入一个新的空闲结点；
入队元素保存到 rear 所指结点中；rear=rear->next；返回。
```

出队操作

```
if（front==rear） // 队空
则出队失败，返回；
取 front 所指结点中的元素 e；front=front->next；返回 e。
```

考点　队列

#280 408真题

408-2026-42

答案

当 n=9时，出栈序列的可能性分析

- 序列{2,3,1,6,4,7,5,8}（假设包含9，即{2,3,1,6,4,7,5,8,9}）：存在下标i=4,j=5,k=7，满足Pj=4<Pk=5<Pi=6，即存在“312”模式，因此不能由栈得到。
- 序列{2,3,1,4,6,5,7,8}（假设包含9，即{2,3,1,4,6,5,7,8,9}）：不存在任何三个下标i<j<k满足Pj<Pk<Pi，因此可以由栈得到。

答案：第一个序列不能得到，第二个序列可以得到。

（2）不能由栈得到的出栈序列中 Pi,Pj,Pk的大小关系

若出栈序列P1,P2,...,Pn不能由栈得到，则存在三个下标i<j<k，满足

Pj<Pk<Pi.即第二个出栈的数最小，第三个出栈的数居中，第一个出栈的数最大。

答案： Pj<Pk<Pi.

（3） n=4时以2开头的出栈序列个数

枚举所有以2开头的出栈序列：

{2,1,3,4},{2,1,4,3},{2,3,1,4},{2,3,4,1},{2,4,3,1}.共5个。

答案： 5个。

（4） n=k时，以1开头、以2开头的序列个数及总个数

已知当n=k−1时，栈可得到的出栈序列总数为

Ck−1=M其中Cn为第n个卡特兰数。

以 1 开头的出栈序列个数

若第一个出栈元素为 1，则操作只能是：

```
push(1) → pop(1)
```

此后对2,3,...,k的出栈过程不再受限制，因此剩余元素的出栈序列个数就是规模为k−1时的总数，即Ck−1=M因此，以 1 开头的出栈序列个数为M。

以 2 开头的出栈序列个数

若第一个出栈元素为 2，则操作前缀必为：

```
push(1), push(2), pop(2)
```

此时栈中剩余元素为 1，尚未入栈的元素为3,4,...,k。

接下来的过程可以看作：在栈中已经保留元素 1 的基础上，继续对3,4,...,k进行正常的入栈、出栈操作。将元素3,4,...,k分别减去 1 后，与规模为k−1的序列1,2,...,k−1的合法出栈过程一一对应，因此合法出栈序列个数同样为

Ck−1=M因此，以 2 开头的出栈序列个数也为M。

出栈序列总数

卡特兰数满足递推关系：

Ck=k+12(2k−1)Ck−1由Ck−1=M，得：

Ck=k+12(2k−1)M所以：

- 以 1 开头的出栈序列个数： M- 以 2 开头的出栈序列个数： M- 出栈序列总数： k+12(2k−1)M

考点　入栈出栈序列、卡特兰数

第五章　数组广义表 2 题

#287 408真题

408-2018-41

答案

1）题目要求算法时间上尽可能高效，因此采用空间换时间的办法。分配一个用于标记的数组 B[n]，用来记录A中是否出现了1~n中的正整数，B[0]对应正整数1，B[n−1]对应正整数n，初始化B中全部为0。由于A中含有n个整数，因此可能返回的值是1~n+1，当A中n个数恰好为1~n时返回n+1。当数组A中出现了小于等于0或大于n的值时，会导致1~n中出现空余位置，返回结果必然在1~n中，因此对于A中出现了小于等于0或大于n的值可以不采取任何操作。

经过以上分析可以得出算法流程：从A[0]开始遍历A，若0<A[i]≤n，则令B[A[i]−1]=1；否则不进行操作。对A遍历结束后，开始遍历数组B，若能查找到第一个满足B[i]=0的下标i，返回i+1即为结果，此时说明A中未出现的最小正整数在1~n之间。若B[i]全部不为0，返回i+1（跳出循环时i=n，i+1等于n+1），此时说明A中未出现的最小正整数是n+1。

2）算法实现

```
int findMissMin(int a[], int n) {
// 来记录 1 ~ n 是否出现过
int vis[n];
for (int i = 0; i < n; i++) {
vis[i] = 0;
}
// 在 vis 数组中记录哪些正整数出现过
for (int i = 0; i < n; i++) {
int num = a[i];
if (num > 0) {
vis[num-1] = 1;
}
}
// 遍历 vis 来找到最小消失的正整数
for (int i = 0; i < n; i++) {
if (vis[i] == 0) {
return i+1;
}
}
return n+1;
}
```

3）时间复杂度：遍历 A一次，遍历B一次，两次循环内操作步骤为O(1) 量级，因此时间复杂度为O(n) 。空间复杂度：额外分配了B[n], 空间复杂度为O(n) 。

考点　顺序表

#288 408真题

408-2025-41

答案

1）若 A[i] 为负数的话，则 A[j] 为子数组 A[i:n] 中的最小值时，A[i] * A[j] 为最大值。若 A[i] 为正数的话，则 A[j] 为子数组 A[i:n] 中的最大值时，A[i] * A[j] 为最大值。

因此算法步骤如下：

1. 从右到左遍历数组 A：维护一个变量 maxValue 来存储当前从 i 到 n-1 范围内的最大值。维护一个变量 minValue 来存储当前从 i 到 n-1 范围内的最小值。在每次迭代中，更新 maxValue 和 minValue。计算 A[i] * maxValue 和 A[i] * minValue，并将它们的较大值存储在 res[i] 中。
2. 维护一个变量 maxValue 来存储当前从 i 到 n-1 范围内的最大值。
3. 维护一个变量 minValue 来存储当前从 i 到 n-1 范围内的最小值。
4. 在每次迭代中，更新 maxValue 和 minValue。
5. 计算 A[i] * maxValue 和 A[i] * minValue，并将它们的较大值存储在 res[i] 中。
6. 更新 maxValue 和 minValue：在每次迭代中，maxValue 是当前元素与之前的 maxValue 的较大值，minValue 是当前元素与之前的 minValue 的较小值。

2）算法实现如下：

```
void calMulMax(int A[], int res[], int n) {
int minValue = INT32_MAX;
int maxValue = INT32_MIN;
for (int i = n-1; i >= 0; i--) {
if (A[i] < minValue) {
minValue = A[i];
}
if (A[i] > maxValue) {
maxValue = A[i];
}
if (A[i] > 0) {
res[i] = A[i] * maxValue;
} else {
res[i] = A[i] * minValue;
}
}
}
```

3）算法的时间复杂度为 O(n)，空间复杂度为 O(1)。

考点　顺序表

第六章　树 8 题

#299 408真题

408-2016-42

答案

1）根据定义，正则 k叉树中仅含有两类结点；叶结点（个数记为n0）和度为k的分支结点（个数记为n1）。树T中的结点总数n=n0+nk=n0+m。树中所含的边数e=n−1，这些边均为m个度为k的结点发出

的，即e=mk。整理得n0+m=mk+1，故n0=(k−1)m+1。

2）高度为 h的正则k叉树T中，含最多结点的树形为：除第h层外，第1层到第h−1层的结点都是度为k的分支结点；而第h层均为叶结点，即树是“满”树。此时第j（1≤j≤h）层结点数为kj−1，结点总数M1为M1=i=0∑hkj−1=k−1kh−1含最少结点的正则K叉树的树形为：第1层只有根结点，第2层到第h−1层仅含1个分支结点和k−1个叶结点，第h层有K个叶结点。即除根外第2层到第h层中每层的结点数均为k，故T中所含结点总数M2为

M2=1+(h−1)k【评分说明】①参考答案仅给出一种推导过程，若考生采用其他推导方法且正确，同样给分。 ②若考生仅给出结果，但没有推导过程，则 (1)、(2) 的最高得分分别是 2 分和 3 分。若推导过程或答案不完全正确，酌情给分。

考点　树的概念

#300　408真题

408-2020-42

答案

1）使用一棵二叉树保存字符集中各字符的编码，每个编码对应于从根开始到达某叶结点的 一条路径，路径长度等于编码位数，路径到达的叶结点中保存该编码对应的字符。

2）从左至右依次扫描 0/1 串中的各位。从根开始，根据串中当前位沿当前结点的左子指针 或右子指针下移，直到移动到叶结点时为止。输出叶结点中保存的字符。然后从根开始重复这个过程，直到扫描到 0/1 串结束，译码完成。

3）二叉树既可用于保存各字符的编码，又可用于检测编码是否具有前缀特性。判定编码是 否具有前缀特性的过程，也是构建二叉树的过程。初始时，二叉树中仅含有根结点，其左子指针和右子指针均为空。

依次读入每个编码 C，建立/寻找从根开始对应于该编码的一条路径，过程如下：

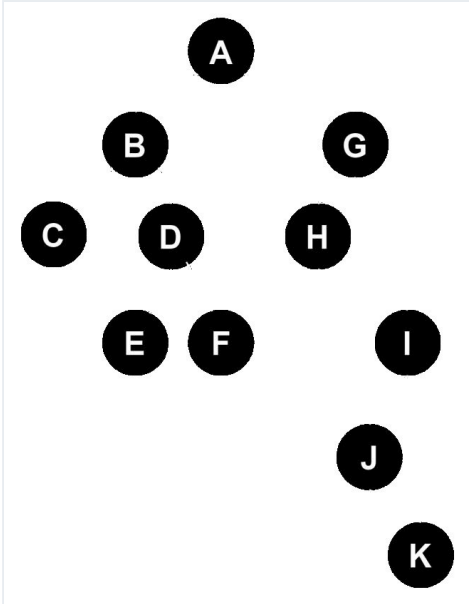
对每个编码，从左至右扫描 C 的各位，根据 C 的当前位 (0 或 1) 沿结点的指针（左子指针或右子指针）向下移动。当遇到空指针时，创建新结点，让空指针指向该新结点并继续移动。沿指针移动的过程中，可能遇到三种情况：

1. 若遇到了叶结点（非根），则表明不具有前缀特性，返回。
2. 若在处理 C 的所有位的过程中，均没有创建新结点，则表明不具有前缀特性，返回。
3. 若在处理 C 的最后一个编码位时创建了新结点，则继续验证下一个编码。

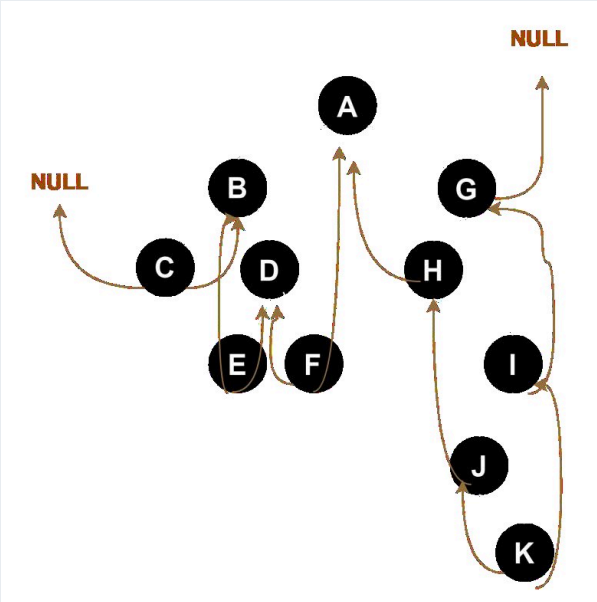
若所有编码均通过验证，则编码具有前缀特性。

考点　前缀编码

#301　802真题2024



图：z2425-p19-0.png



图：z2425-p19-1.png

答案

（1）补全： 先序序列：A B C D E F G H I J K 中序序列：C B E D F A H J K I G 后序序列：C E F D B K J I H G A （2）二叉树结构：

```
        A
      /   \
     B     G
    / \   /
   C   D H
      / \  \
     E   F  I
          /
         J
          \
           K
```

（3）中序线索链表：按中序 C B E D F A H J K I G 建立线索(见答案PDF图)。

解析

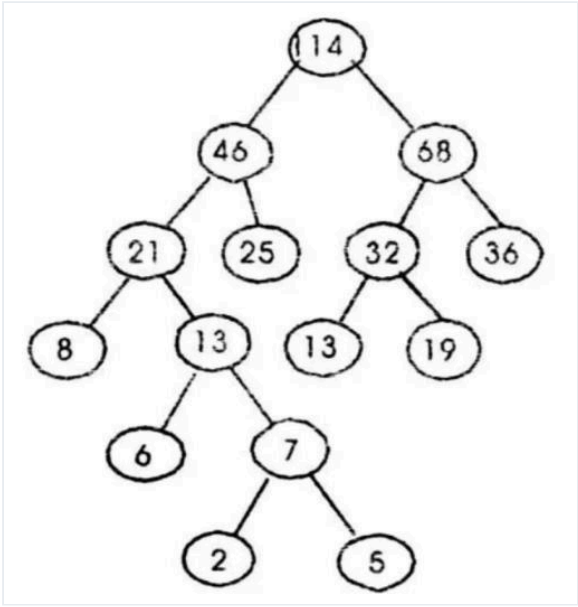
先序定根、中序划分左右子树递归重建。由先序首 A 为根、中序中 A 分左右；逐层确定 B(左:C, 右:D(E,F))、G(左:H(右:I(左:J(右:K))))。三序列补全后经交叉验证完全自洽，后序推导与答案一致。

备注：树形图与中序线索链表图见答案PDF P19(z2425-p19-0.png, z2425-p19-1.png)。

考点　二叉树三序遍历、由遍历序列构造二叉树、中序线索链表

#302　802真题2024

PDF P5　·　真题-2024-综合5



图：z2425-p21-0.png

答案

正确的 Huffman 编码(一种合法最优方案，0/1 左右约定可整体互换)： g=0, d=100, e=110, f=111, c=1010, a=10110, b=10111 对应带权路径长度 WPL = 36×1 + 8×3 + 13×3 + 19×3 + 6×4 + 2×5 + 5×5 = 215(最优)。 构造过程：反复合并最小两权 —— 2+5=7, 6+7=13, 8+13=21, 13+19=32, 21+32=53, 36+53=89。

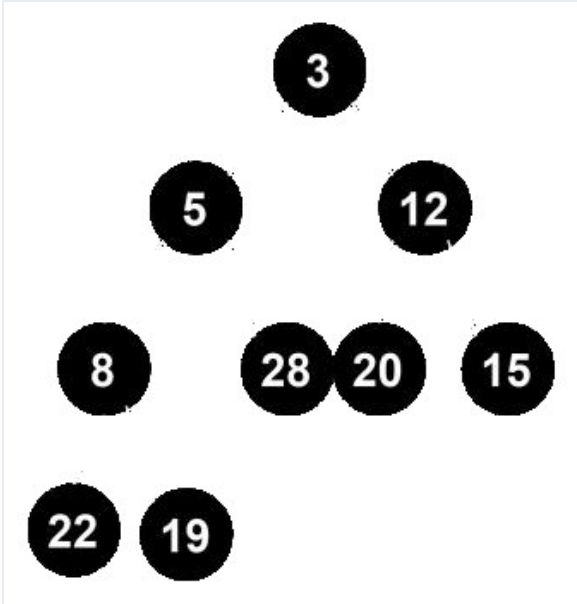
解析

Huffman 每次取当前最小两权合并；频率最大的 g(36) 应最早停留在最后一步合并、码长最短(=1)。WPL 最小为 215。

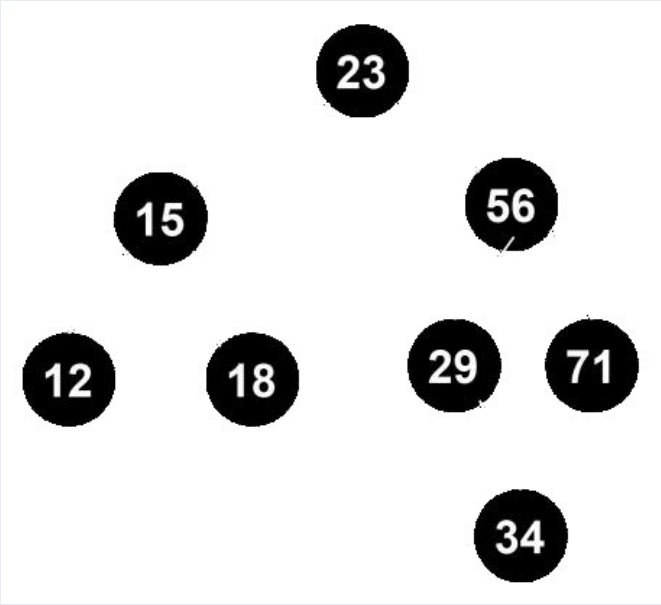
【勘误•以修正版为准】 【答案PDF有误】PDF 给出编码 a=00110,b=00111,c=0010,d=000,e=100,f=101,g=11，存在两处问题：(1) g(频率最大 36) 码长为 2，而任何 Huffman 树中最大权必位于最短码长(应为 1)，该编码 WPL=251 > 最优 215，非最优；(2) 前缀 '01' 分支为空，不是满二叉树，结构非法。正确 WPL=215，编码见 answer(哈夫曼树图见答案PDF P21 z2425-p21-0.png，但该图对应错误编码，应据正确构造重画)。

考点　哈夫曼树、哈夫曼编码、WPL

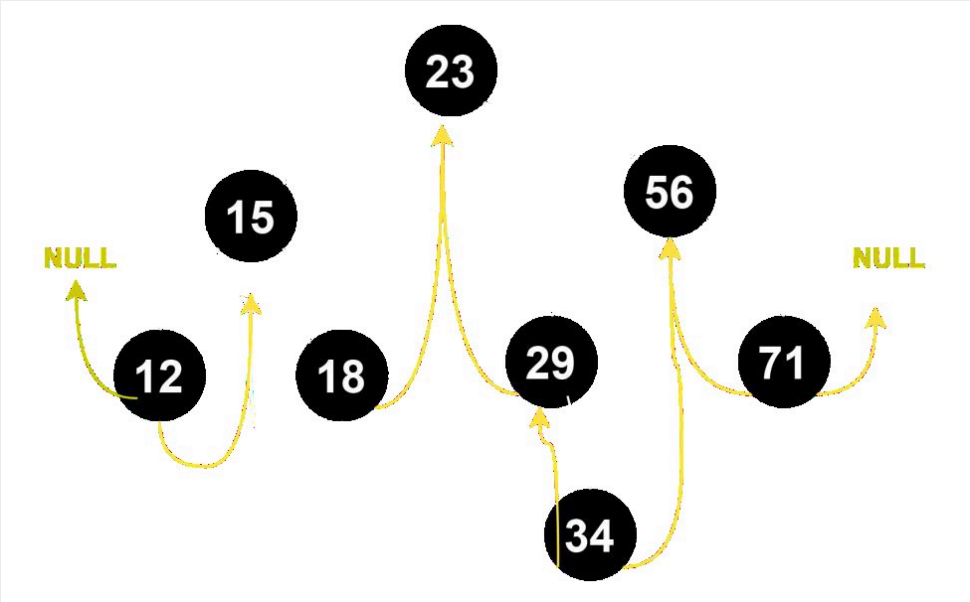
#303　802真题2025



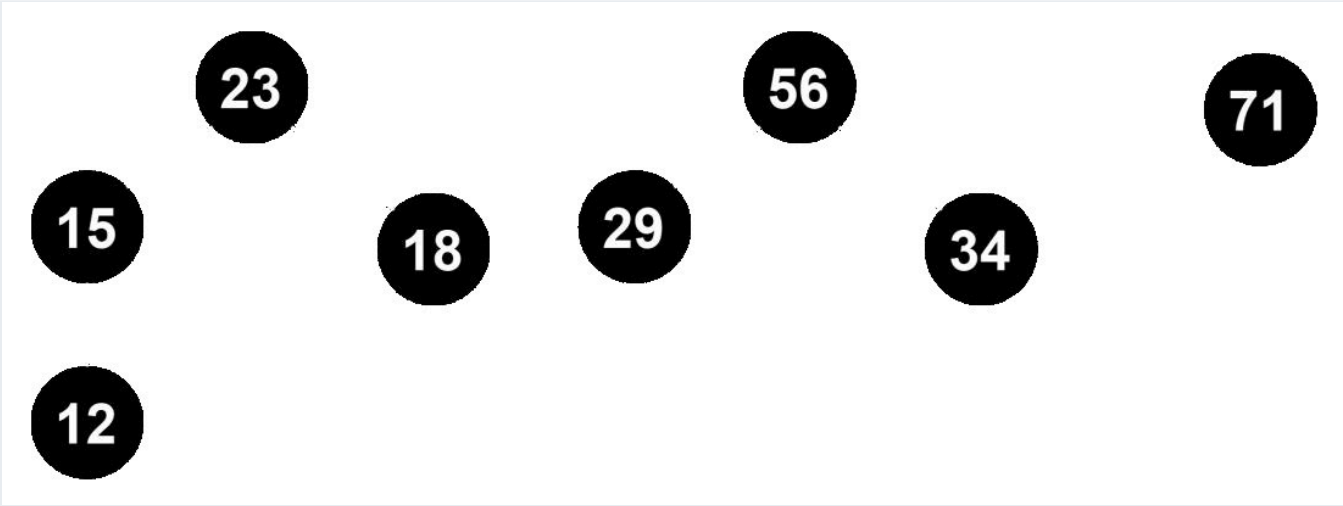
图：z2425-p32-0.png



图：z2425-p32-1.png



图：z2425-p32-2.png



图：z2425-p33-0.png

答案

（1）BST 形态（先序定根、按大小分左右）： 23 / \ 15 56 / \ / \ 12 18 29 71 \ 34 （2）后序遍历序列：12, 18, 15, 34, 29, 71, 56, 23。 （3）中序序列为 12,15,18,23,29,34,56,71（升序）；据此把空左链改为指向前驱、空右链改为指向后继的线索，即得中序线索二叉树（图见答案PDF）。 （4）二叉树转森林：根 23 无右兄弟故对应一棵树；按'左孩子右兄弟'还原，得对应森林（图见答案PDF）。

解析

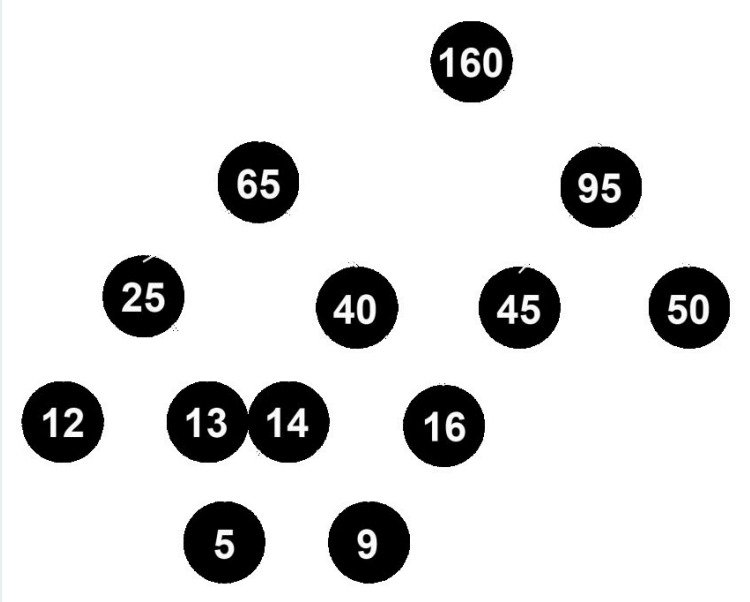
先序第一个 23 为根；BST 中小于 23 的 {15,12,18} 在左子树、大于 23 的 {56,29,34,71} 在右子树，递归重建。左子树先序 15,12,18→15 为根、12 左、18 右；右子树先序 56,29,34,71→56 为根，{29,34} 左、71 右，29 的右孩子为 34。中序即升序序列。后序=左后序+右后序+根=12,18,15 + 34,29,71,56 + 23。

备注：答案PDF已核对：后序序列与按 BST 重建结果完全一致，正确。(1)(3)(4) 图形见答案PDF图 z2425-p32-*.png、z2425-p33-0.png。

考点　二叉排序树BST、先序重建、后序遍历、中序线索二叉树、二叉树与森林转换

#304 802真题2025

PDF P11 · 真题-2025-综合3



图：z2425-p34-0.png

答案

哈夫曼编码（左0右1）： a(5)=0100 b(9)=0101 c(12)=000 d(13)=001 e(16)=011 f(45)=10 g(50)=11 构造过程(每次取最小两权合并，左≤右)：5+9=14；12+13=25；14+16=30；25+30=55；45+50=95；55+95=150(根)。

解析

合并序列得树：根150(左55,右95)；55(左25,右30)；25(左c12,右d13)；30(左14,右e16)；14(左a5,右b9)；95(左f45,右g50)。按左0右1从根到叶即得各字符编码，与答案完全一致。WPL=150（各叶权×码长之和，可选核对）。

备注：已逐字符核对编码，答案PDF正确。哈夫曼树形见答案PDF图 z2425-p34-0.png。

考点 哈夫曼树、哈夫曼编码、WPL

#305 802真题2026

PDF P4 · 真题-2026-综合16

答案

（1）后序序列：BDFGECA。树形：A 为根，左孩子 B，右子树 C(D, E(F,G))。 （2）三叉链表：在二叉链表基础上增加 parent 指针指向父结点。 （3）对应森林由 4 棵树组成：(A-B)、(C-D)、(E-F)、(G)。

解析

先序定根、中序划分左右子树，递归重建。

考点 二叉树重建、三序遍历、三叉链表、树森林转换

#306 802真题2026

PDF P5 · 真题-2026-综合22

备注：【数据缺失】回忆版注明各字符出现次数'(略)'，未给出具体权值。答案 PDF 注：'没给具体权值，但哈夫曼树的题目一般不难'。仅作真题登记。

考点 哈夫曼树、哈夫曼编码

第七章　图 13 题

#314 408真题

408-2015-42

答案

1）图 A的邻接矩阵如下：

A=0110110011100100110111010 2） A2如下：

A=310311321202202310311221 30 行 3 列的元素值 3 表示从顶点 0 到顶点 3 之间长度为 2 的路径共有 3 条。

3） Bm(2≤m≤n) 中位于i行j列（0≤i,j≤n−1）的非零元素的含义是：图中从顶点i到顶点j长度为m的路径条数。

考点　邻接矩阵

#315 408真题

408-2017-42

答案

1）Prim 算法 属于贪心策略。算法从一个任意的顶点开始，一直长大到覆盖图中所有顶点为止。算法每一步在连接树集合 S 中顶点和其他顶点的边中，选择一条使得树的总权重增加最小的边加入集合 S。当算法终止时，S 就是最小生成树。

1. S 中顶点为 A，候选边为 (A,D),(A,B),(A,E)，选择 (A,D) 加入 S。
2. S 中顶点为 A,D，候选边为 (A,B),(A,E),(D,E),(C,D)，选择 (D,E) 加入 S。
3. S 中顶点为 A,D,E，候选边为 (A,B),(C,D),(C,E)，选择 (C,E) 加入 S。
4. S 中顶点为 A,D,E,C，候选边为 (A,B),(B,C)，选择 (B,C) 加入 S。
5. S 就是最小生成树。

依次选出的边为 (A,D),(D,E),(C,E),(B,C)（4 分）

【评分说明】每正确选对一条边且次序正确，给 1 分。若考生选择的边正确，但次序不完 全正确，酌情给分。

2）图 G 的 MST 是唯一的。（2 分）第一小题的最小生成树包括了图中权值最小的四条边，其他边都比这四条边大，所以此图的 MST 唯一。

3）当带权连通图的任意一个环中所包含的边的权值均不相同时，其 MST 是唯一的。（2 分）此题不要求回答充分必要条件，所以回答一个限制边权值的充分条件即可。

【评分说明】①若考生答案中给出的是其他充分条件，例如“带权连通图的所有边的权值均不相同”，同样给分。

②若考生给出的充分条件对图的顶点数和边数做了某些限制，例如，限制了图中顶点的个数（顶点个数少于 3 个）、限制了图的形状（图中没有环）等，则最高给 1 分。

③答案部分正确，酌情给分。

考点　最小生成树

#316 408真题

408-2018-42

答案

1）为了求解最经济的方案，可以把问题抽象为求无向带权图的最小生成树。可以采用手动 Prim 算法或 Kruskal 算法作图。注意本题最小生成树有两种构造，如下图所示。

XABJWHCSQDTLJNNJ2222233XABJWHCSQDTLJNNJ2222233方案 1方案 2方案的总费用为 16。

2）存储题中的图可以采用邻接矩阵（或邻接表）。构造最小生成树采用 Prim 算法（或 Kruskal 算法）。

3）TTL= 5，即 IP 分组的生存时间（最大传递距离）为 5，方案 1 中 TL 和 BJ 的距离过远，TTL = 5 不足以让 IP 分组从 H1 传送到 H2，因此 H2 不能收到 IP 分组。而方案 2 中 TL 和 BJ 邻近，H2 可以收到 IP 分组。

考点　最小生成树、邻接表、路由器

#317　408真题

408-2021-41

答案

1）算法的基本设计思想

本算法题属于送分题，题干已经告诉我们算法的思想。对于采用邻接矩阵存储的无向图，在邻接矩阵的每一行（列）中，非零元素的个数为本行（列）对应顶点的度。可以依次计算连通图 G 中各顶点的度，并记录度为奇数的顶点个数，若个数为 0 或 2，则返回 1，否则返回 0。

2）算法实现

```
int isExistEL(MGraph G) {
// 统计度为奇数的定点个数
int count = 0;
for (int v = 0; v < numVertices; v++) {
// 该顶点的度
int degree = 0;
for (int e = 0; e < numEdges; e++) {
if (Edge[v][e] == 1) {
degree++;
}
}
if (degree % 2 == 1) {
count++;
}
}
if (count == 0 || count == 2) {
return 1;
}
return 0;
}
```

3）时间和空间复杂度

算法需要遍历整个矩阵，所以时间复杂度为O(n2)，空间复杂度为O(1)。

考点　邻接矩阵

#318　408真题

408-2023-41

答案

1）采用邻接矩阵表示有向图时，一行中 1 的个数为该行对应顶点的出度，一列中 1 的个数为该列对应顶点的入度。使用一个初值为 0 的计数器记录 K 顶点的个数。对图 G 的每个顶点，根据邻接矩阵计算其出度 outdegree 和入度 indegree。若 outdegree - indegree > 0，则输出该顶点且计数器加 1。最后返回计数器的值。

2）算法实现

```
int printVertices(MGraph G) {
// K 顶点的个数
int count = 0;
for (int v = 0; v < numVertices; v++) {
// v 顶点的入度和出度
int indegree = 0;
int outdegree = 0;
// 统计出度
for (int i = 0; i < numVertices; i++) {
outdegree += G.Edge[v][i];
}
// 统计入度
for (int i = 0; i < numVertices; i++) {
indegree += G.Edge[i][v];
}
if (outdegree > indegree) {
printf("%s ", G.VerticesList[v]);
count++;
}
}
return count;
}
```

考点　邻接矩阵

#319　408真题

408-2024-41

答案

1）算法基本设计思想

输出拓扑序列的过程如下：依次从图中选取入度为 0 的点进行输出。

确保拓扑序列唯一需要保证如下条件：在输出拓扑序列时，每一次有且仅有一个入度为 0 的顶点。

所以这题最直观的思路就是进行 numEdges 轮遍历，如果每轮遍历只有一个顶点的入度为 0，则图中存在唯一的拓扑序列； 否则不存在唯一的拓扑序列。

2）算法实现

```
int uniquely(MGraph g)
{
int n = g.numVertices;
int inDegrees[MAXV];

/* 计算所有顶点的入度 */
for (int v = 0; v < n; v++) {
inDegrees[v] = 0;
```

```
for (int i = 0; i < n; i++) {
if (g.Edge[i][v] != 0)
inDegrees[v] += g.Edge[i][v];
}
}

/* 进行 n 轮拓扑删除 */
for (int k = 0; k < n; k++) {

int count0 = 0;   // 入度为 0 的顶点个数
int v0 = -1;      // 本轮唯一可以选择的顶点

/* 找所有入度为 0 的顶点 */
for (int i = 0; i < n; i++) {
if (inDegrees[i] == 0) {
count0++;
v0 = i;
}
}

/* 若不是唯一一个，则拓扑序列不唯一 */
if (count0 != 1) {
return 0;
}

/* “删除”该顶点：将其入度设为 -1，避免重复选取 */
inDegrees[v0] = -1;

/* 更新其所有后继节点的入度 */
for (int j = 0; j < n; j++) {
if (g.Edge[v0][j] != 0)
inDegrees[j]--;
}
}

/* 所有步骤都唯一，拓扑序唯一 */
return 1;
}
```

考点　拓扑排序、邻接矩阵

#320 408真题

408-2025-42

答案

1）首先计算最早发生时间 ve：

| node | 1 | 2 | 3 | 4 | 5 | 6 | 7 || --- | --- | --- | --- | --- | --- | --- | --- || ve | 0 | 9 | 2 | 5 | 12 | 6 | 9 |

然后计算最晚发生时间：

| node | 1 | 2 | 3 | 4 | 5 | 6 | 7 || --- | --- | --- | --- | --- | --- | --- | --- || vl | 0 | 10 | 2 | 5 | 12 | 8 | 9 |

ve 和 vl 相同的顶点为 1、3、4、7、5，所以关键活动为 a、e、m、n。

2）e 的执行时间是 2~5，可以与 e 同时执行的活动为 b、d、c。

3）假设有顶点 i→j 的活动（边），则该活动的 时间余量 = vl(j) - ve(i) - 边的长度。

计算所有非关键路径上活动的时间余量，可以得到活动 j 的时间余量是最大的，为 vl(5) - ve(4) - 1 = 6。

4）为保证 1 → 2 → 5 的路径长度不超过关键路径的长度，需要保证 6 + b + k ≤ 12，所以 b ≤ 4，即 b 的持续时间最长为 4。如果想要保证 b 仍然为 5 的话，需要压缩 k 的时间长度，将 k 缩小为 1。

考点　关键路径

#321　802真题2024

PDF P5　·　真题-2024-综合4

答案

按 Floyd 算法依次以 V0..V4 为中转点迭代，逐步更新距离矩阵 D(0)→D(4) 与路径矩阵 Path(0)→Path(4)；每步 $D(k)[i][j]=\min(D(k-1)[i][j], D(k-1)[i][k]+D(k-1)[k][j])$，若更新则 $Path[k][i][j]=k$。

解析

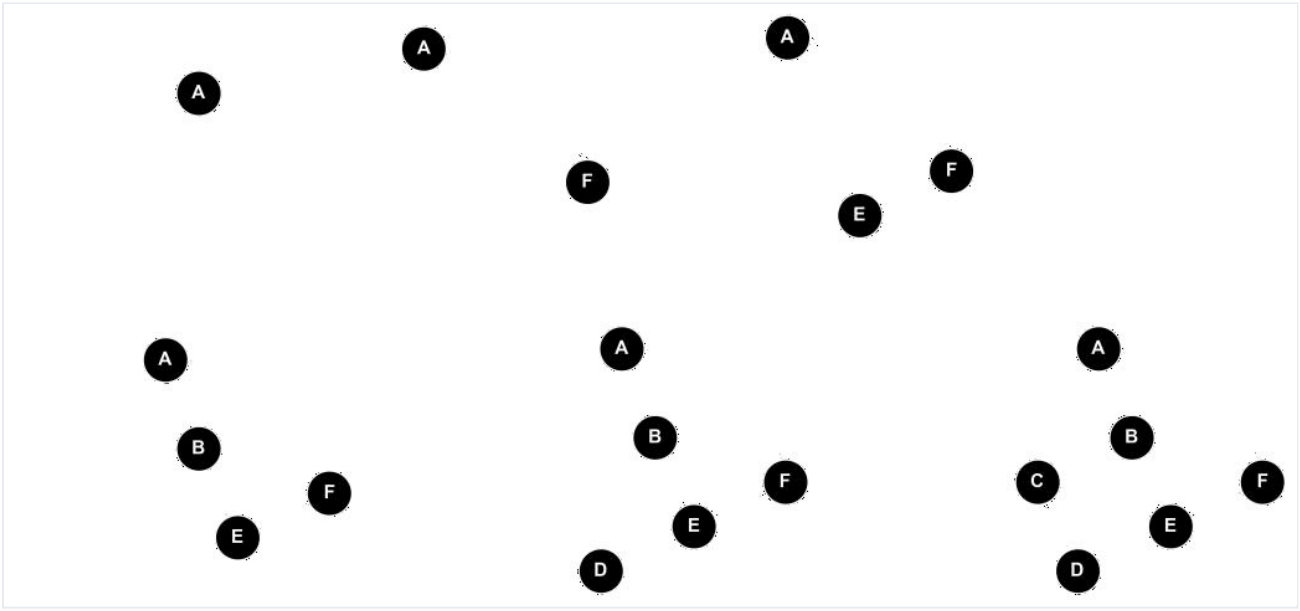
Floyd 三重循环 $O(V^3)$，逐个允许新中转顶点刷新所有点对最短距离，Path 记录中转点用于回溯具体路径。

备注：【图/数据缺失】题干邻接矩阵文本在回忆版OCR中严重错乱(数字与 V0..V4 标号散落无法还原)，图的完整信息以附图 z2425-p05-0.png 为准；答案PDF P20-P22 给出的 D(0)~D(4)、Path 矩阵同样被OCR打散无法可靠转录。此处登记考点与算法过程，具体矩阵数值需据附图重算。

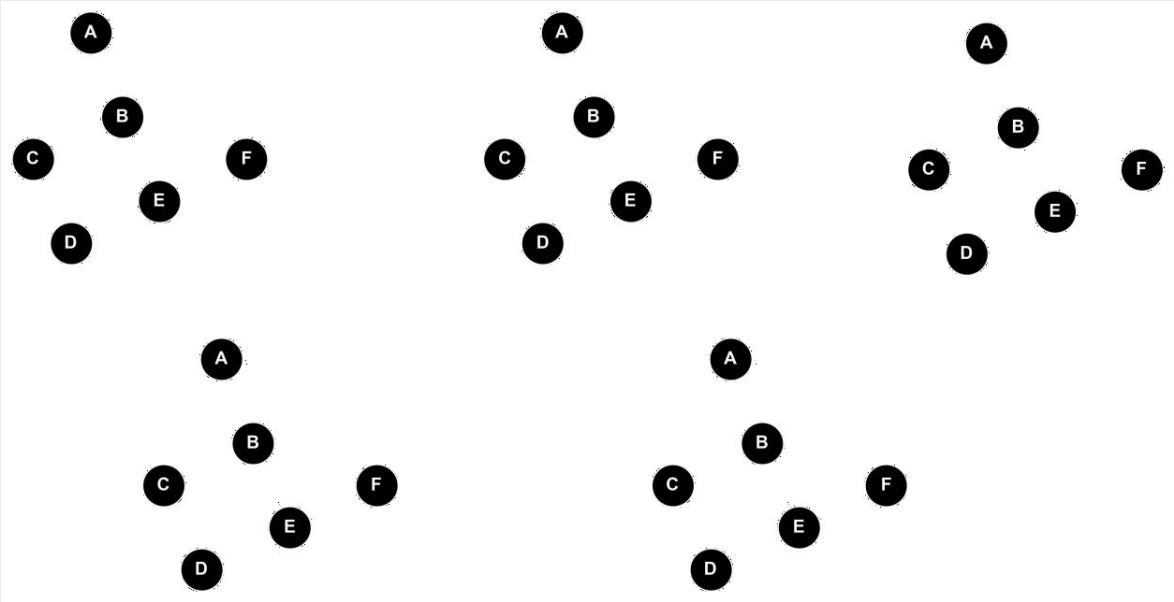
考点　Floyd最短路径、距离矩阵D、路径矩阵Path

#322　802真题2025

PDF P11　·　真题-2025-综合5



图：z2425-p36-0.png



图：z2425-p36-1.png

答案

（1）BFS 序列：A B F C D E；DFS 序列：A B C D E F。 （2）Prim(从 A 起)与（3）Kruskal 得到的最小生成树见答案PDF附图（z2425-p36-0.png、z2425-p36-1.png）。

解析

BFS 按层扩展、DFS 一路深入，均在权值/多分支相同时按字母序小者优先。Prim 从 A 逐步并入最近顶点；Kruskal 按边权从小到大选不成环的边。具体 MST 边集依赖图中权值。

备注：题干依赖'完全无向图 G'的图。P11 有两张题图(z2425-p11-0.png 301x298、z2425-p11-1.png 677x345)，分别对应综合5的完全无向图 G 与综合6的 AOE 网，因无图内文本无法从 OCR 精确区分二者对应关系，故两图一并附上。BFS/DFS 序列取自答案PDF；MST 结果为答案PDF附图。

考点 图遍历BFS/DFS、最小生成树、Prim、Kruskal

#323 802真题2025

PDF P11 · 真题-2025-综合6

答案

（1）事件 ve / vl（顶点 a,b,c,d,e,f,h,g,i）： ve: a=0, b=6, c=4, d=5, e=7, f=16, h=7, g=14, i=19 vl: a=0, b=6, c=6, d=8, e=7, f=17, h=10, g=14, i=19 （2）活动 e(ai) / l(ai)（活动 ab,ac,ad,be,ce,ef,eg,dh,hg,fi,gi）： e: 0,0,0,6,4,7,7,5,7,16,14 l: 0,2,4,6,6,8,7,8,10,17,14 关键活动(e==l)：ab, be, eg, gi。 （3）关键路径：a → b → e → g → i（长度 ve(i)=19）。

解析

ve 从源点向后取最大、vl 从汇点向前取最小；活动 e(ai)=ve(弧尾)，l(ai)=vl(弧头)-权值。e==l 的活动为关键活动，串成关键路径 a→b→e→g→i，权值和 6+1+7+5=19=ve(i)，自洽。

备注：答案已核对：关键活动 ab,be,eg,gi 满足 e==l，关键路径 a→b→e→g→i 长度 19 与 ve(i) 一致，正确。AOE 网图见 P11 题图(z2425-p11-*.png，与综合5共用两图，对应关系需查原图)。

考点 AOE网、关键路径、ve/vl、e/l、关键活动

#324 802真题2026

PDF P4 · 真题-2026-综合19

备注：【图/题面缺失】回忆版仅存一句题干与一张不清晰的图(z26-p04-0.png，464x258)。答案 PDF 注：'图不清晰，甚至不知道是单源点到其他节点还是结点之间，但可根据 Dijkstra 或 Floyd 计算'。仅作真题登记，无法结构化作答。

考点　最短路径、Dijkstra/Floyd

#325 802真题2026

PDF P4 · 真题-2026-综合20

备注：【图缺失】回忆版给出了表格表头(顶点/活动)但未给出 AOE 网的图与边权。答案 PDF 注：'没有给图，做不了，但按从前向后求最大、从后往前求最小可得事件最早最晚，再求活动的最早最晚'。仅作真题登记。

考点　AOE网、关键路径、ve/vl/e/l

#326 802真题2026

PDF P5 · 真题-2026-综合21

备注：【图缺失】回忆版给出表格框架但未给出有向图。答案 PDF 注：'没给图，但这个题和 19 题思路很像，可能有一个题有问题、记重复了，应该不会一张卷子考两次相同算法'。仅作真题登记。

考点　Dijkstra、最短路径表格

第八章　查找 11 题

#333 408真题

408-2022-41

答案

1）算法的基本设计思想

对于采用顺序存储方式保存的二叉树，根结点保存在 SqBiTNode[0] 中：当某结点保存在 SqBiTNode[i] 中时，若有左孩子，则其值保存在 SqBiTNode[2i+1]中；若有右孩子，则其值保存在 SqBiTNode[2i+2] 中；若有双亲结点，则其值保存在 SqBiTNode[(i-1)/2] 中。

二叉搜索树需要满足的条件是：任一结点值大于其左子树中的全部结点值，小于其右子树中的全部结点值。中序遍历二叉搜索树得到一个升序序列。

使用整型变量 val 记录中序遍历过程中已遍历结点的最大值，初值为一个负整数，对二叉树进行中序遍历。若当前遍历的结点值小于等于 val，则算法返回 false，否则，将 val 的值更新为当前结点的值。

2）算法实现

```
// val 存储中序遍历中访问到的最大值
// 返回值：当前子树是否为 BST
bool solve(SqBiTree *tree, int k, int *val) {
if (k >= tree->ElemNum) {
// 空结点
return true;
}
// 判断左子树是否为 BST
bool ret = solve(tree, 2*k+1, val);
if (!ret) {
return false;
}
int cur_val = tree->SqbiTNode[k];
if (cur_val == -1) {
```

```
// 空结点
return true;
}
// 判断中序序列是否递增
if (cur_val > *val) {
*val = cur_val;
} else {
return false;
}
// 判断右子树是否为 BST
ret = solve(tree, 2*k+2, val);
if (!ret) {
return false;
}
return true;
}
```

考点　二叉排序树

#334　408真题

408-2022-42

答案

【答案 1】

定义含 10 个元素的数组 A，初始时元素值均为该数组类型能表示的最大数 MAX。

```
for M 中的每个元素 s
if (s < A[9]) 丢弃 A[9]并将 s 按升序插入到 A 中；
```

当数据全部扫描完毕，数组 A[0]~A[9]保存的即是最小的 10 个数。

【答案 2】

定义含 10 个元素的大根堆 H，元素值均为该堆元素类型能表示的最大数 MAX。

```
for M 中的每个元素 s
if （s < H 的堆顶元素）
```

删除堆顶元素并将 s 插入到 H 中； 当数据全部扫描完毕，堆 H 中保存的即是最小的 10 个数。

考点　堆的概念、插入排序

#335　408真题

408-2024-42

答案

1）散列表 HT 如下：

| | | | | | | | | | | | | || --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- || 散列地址 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 || 关键字 | 11 | | | 14 | 7 | | | 20 | 9 | | | 3 | 18 | | | || 冲突次数 | 1 | | | 3 | 2 | | | 1 | 2 | | | 1 | 1 | | | |

装填因子等于散列表中已经被填充的位置的数量除以散列表的总长度，因此本题的填装因子是 7/11。

2）查找关键字 14 的比较序列：

1. 首先，我们计算 14 的散列地址： H(14)=(14×3)%11=42%11=9。
2. 我们查看散列表中索引为 9 的位置，发现哪里存储的是关键字 3，此时产生哈希冲突。

3. 由于我们使用的是二次探查，所以计算下一个散列地址： H1=(H0+11)%11=(9+1)%11=10。 发现那哪里存储的是关键字 18，再次遇到哈希冲突。
4. 继续计算下一个散列地址： H2=(H0+22)%11=(9+4)%11=2，找到关键字 14，

3）查找关键字 8 失败时的哈希地址

1. 计算 8 的散列地址： H(8)=(8×3)%11=24%11=2，索引为 2 的位置发现关键字 18，遇到冲突。
2. 使用二次探查，计算下一个散列地址： H1=(H0+11)%11=(2+1)%11=3。索引为 3 的位置存储的是关键字 7，遇到冲突。
3. 我们继续使用一次探查，计算下一个散列地址： H2=(H0+22)%11=(2+4)%11=6，索引为 6 的位置存储的是关键字 9，遇到冲突。
4. 我们继续使用二次探查，计算下一个散列地址： H3=(H0+32)%11=(2+9)%11=0。索引为 0 的位置存储的是关键字 11，遇到冲突。
5. 我们继续使用二次探查，计算下一个散列地址： H4=(H0+42)%11=(2+16)%11=7，发现索引为 7 的位置是空的，确认查找失败，散列地址是 7。

考点　散列表

#336　408真题

408-2026-41

答案

算法设计思想：由于二叉搜索树的中序遍历序列为递增序列，本算法采用中序递归遍历二叉树，并记录当前找到的最小绝对值差值 min。遍历过程中，每访问一个结点时计算目标值与当前结点值之差的绝对值：

- 若该值小于 min，则更新 min；
- 若该值大于或等于 min，说明后续结点的差值会更大，此时可停止查找（可通过标志变量 flag 控制递归终止）。最后输出所有差值为 min 的结点。

  注意：题目要求输出与目标值差的绝对值最小的所有结点，可能不止一个结点。例如下图中：结点 5、15 与目标值 10 的差的绝对值均为 5，此时应全部输出。满足条件的结点最多只可能有两个。

```
10
/  \
5   15
```

(2) 算法实现：

```
// 当前的绝对值差值最小值
int min = INT_MAX;
// 最小绝对值差值是否已经找到
int flag = 0;
// 存储待输出的结点关键字
int min_data[2];
int min_idx = 0;

void searchMinDiff(BTNode *root, int K) {
if (!root) return;
if (flag) return;
```

```
searchMinDiff(root->left, K);
// 中序遍历
int diff = abs(K - root->data);
if (diff < min) {
min = diff;
min_data[0] = root->data;
min_idx = 1;
} else if (diff == min) {
min_data[min_idx++] = root->data;
} else {
flag = 1;
}
searchMinDiff(root->right, K);
}

void solve(BTNode *root, int K) {
searchMinDiff(root, K);
printf("min diff: %d\n", min);
for (int i = 0; i < min_idx; i++) {
printf("min element: %d\n", min_data[i]);
}
}
```

考点　二叉排序树

#337　802真题2024

PDF P4　·　真题-2024-综合2

答案

散列表(下标→关键字)：0→11, 1→空, 2→13, 3→24, 4→空, 5→5, 6→28, 7→72, 8→16, 9→18, 10→7。 各关键字查找成功比较次数(按地址序 11,13,24,5,28,72,16,18,7)：1, 1, 2, 1, 1, 2, 4, 3, 4。 ASL成功 = (1+1+2+1+1+2+4+3+4)/9 = 19/9 ≈ 2.11。 各地址查找失败比较次数(地址0..10)：2, 1, 3, 2, 1, 8, 7, 6, 5, 4, 3。 ASL失败 = (2+1+3+2+1+8+7+6+5+4+3)/11 = 42/11 ≈ 3.82。

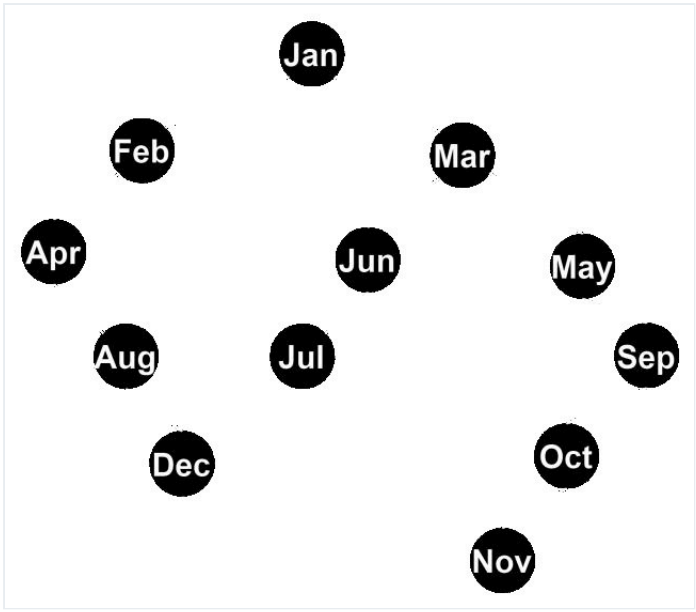
解析

线性探测：H(key)=key%11，冲突则向后逐个探测至空位。查找成功 ASL=各关键字比较次数之和/关键字数；查找失败 ASL=从每个地址探测到首个空位的比较次数之和/表长(11)。空位为地址 1 和 4。

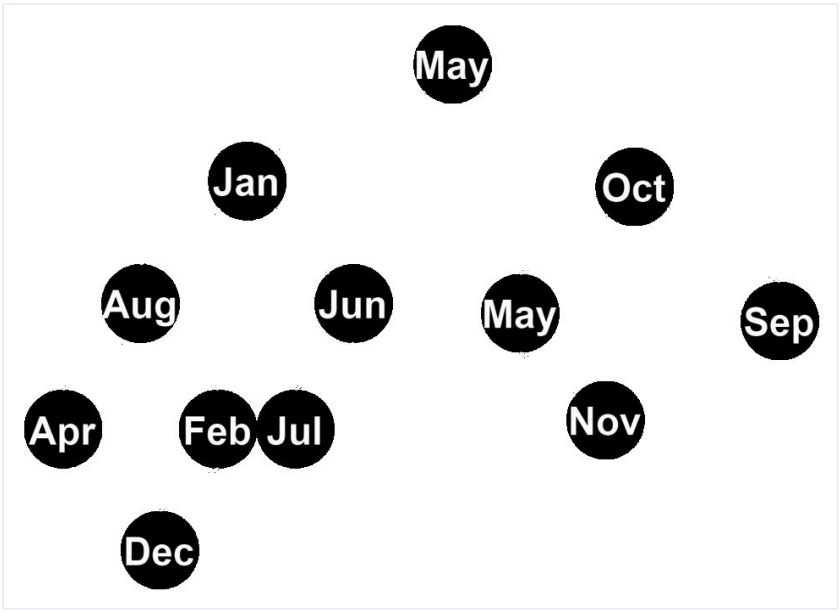
【勘误•以修正版为准】 【答案PDF有误】ASL成功 PDF 写作 (1+1+2+1+3+3+4+1+2)/9 = 2，其求和项与它自己表格中的成功比较次数(1,1,2,1,1,2,4,3,4)不一致，且和也算错(表内实际求和=19)。正确 ASL成功 = 19/9 ≈ 2.11。ASL失败 = 42/11 正确无误。

考点　散列表、线性探测再散列、ASL成功失败

#338　802真题2024



图：z2425-p21-1.png



图：z2425-p22-0.png

答案

（1）按给定顺序插入建 BST，各结点层数：Jan1；Feb,Mar2；Apr,May,Jun3；Jul,Aug,Sep4；Oct,Dec5；Nov6。 ASL成功 = (1×1+2×2+3×3+4×3+5×2+6×1)/12 = 42/12 = 7/2 = 3.5。 （2）排序后 ["Apr","Aug","Dec","Feb","Jan","Jul","Jun","Mar","May","Nov","Oct","Sep"]，折半判定树各层结点数 1,2,4,5。 ASL成功 = (1×1+2×2+3×4+4×5)/12 = 37/12 ≈ 3.08。 （3）构造 AVL，各层结点数 1,2,4,4,1。 ASL成功 = (1×1+2×2+3×4+4×4+5×1)/12 = 38/12 = 19/6 ≈ 3.17。

解析

BST 按插入顺序建、ASL=各结点层数加权平均；折半查找按 mid=(low+high)/2 建判定树；AVL 插入时旋转保持平衡。第(1)(2)问已逐结点/逐层核对无误；第(3)问公式自洽(层结点数 1+2+4+4+1=12)。

备注：BST 与 AVL 树形图见答案PDF P21-P22(z2425-p21-1.png, z2425-p22-0.png)。

考点 二叉排序树BST、折半查找判定树、平衡二叉树AVL、ASL

#339 802真题2025

PDF P10 · 真题-2025-综合2

答案

（1）按答案PDF，各关键码插入地址与探测次数： 2→地址2(探测1)；8→地址8(探测1)；31→地址5(探测1)；20→地址7(探测1)；70→地址12(探测2)；59→地址1(探测2)；25→地址10(探测4)；28→地址4(探测4)。 散列表 HT[0..12]： 地址 0:空 1:59 2:2 3:空 4:28 5:31 6:空 7:20 8:8 9:空 10:25 11:空 12:70 （2）ASL成功=(1+1+1+1+2+2+4+4)/8=16/8=2。

解析

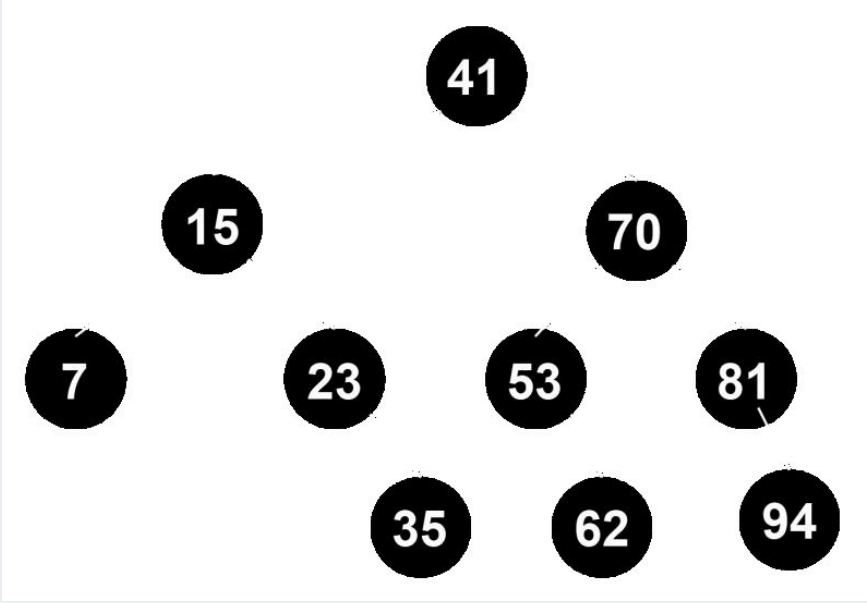
双散列：冲突时用第二个散列函数(基于 reverse(key) 的步长)计算增量继续探测。ASL成功=各元素探测次数之和/元素个数=16/8=2。

备注：ASL成功=2 与探测次数(1,1,1,1,2,2,4,4)自洽。答案PDF部分中间探测算式（如 70 的 H1）与所给公式 reverse(70)=07=7 反推的地址不完全吻合，疑为答案PDF书写/OCR 的算式笔误，但最终落位表与 ASL=2 一致，予以保留；如需精确复算请以公式逐步代入核对。

考点 散列表、双散列法、冲突处理、ASL

#340 802真题2025

PDF P11 · 真题-2025-综合4



图：z2425-p35-0.png

答案

判定树(mid=⌊(low+high)/2⌋)： 41 / \ 15 70 / \ / \ 7 23 53 81 \ \ \ 35 62 94 （2）查找 53：41 → 70 → 53（3 次比较，命中）。 （3）查找 90：41 → 70 → 81 → 94（90>81 右、90<94 左落到空，比较 4 次失败）。 （4）ASL成功=29/10=2.9；ASL失败=39/11≈3.55。

解析

各元素命中比较次数=所在层数：41(1)；15,70(2)；7,23,53,81(3)；35,62,94(4)。ASL成功=(1×1+2×2+4×3+3×4)/10=(1+4+12+12)/10=29/10=2.9。失败结点(外部结点)共 11 个，各失败比较次数之和=39，ASL失败=39/11≈3.55。

【勘误·以修正版为准】 【答案PDF有误】答案PDF给 ASL成功=33/10=3.3，但其自身判定树(与查找53='41→70→53'、查找90='41→70→81→94'及 ASL失败=39/11 完全对应)按层数求和应为

(1×1+2×2+4×3+3×4)/10=29/10=2.9。故 ASL成功正确值为 29/10=2.9，答案PDF的 33/10 有误。ASL失败=39/11 已核对正确。判定树图见 z2425-p35-0.png。

考点　折半查找、判定树、ASL成功失败

#341　802真题2026

PDF P4　·　真题-2026-综合17

答案

（1）线性探测：19→6,14→1,23→10,1→2(冲突探测),68→3,20→7,84→8(6冲突→7→8),27→4(1→2→3→4),55→5(3→4→5),11→11,10→12(10→11→12),79→9(1→2...→8均冲突→9)。（具体 ASL 见答案 PDF 计算）

解析

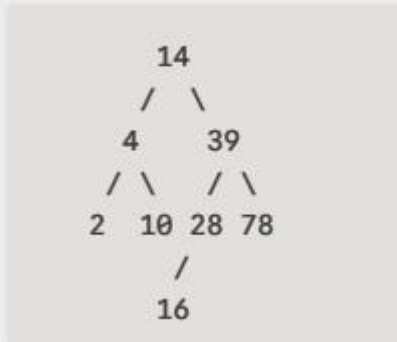
线性探测逐个向后探测空位；链地址法同义词挂同一链表。ASL 成功=各元素比较次数之和/关键字数，失败=各地址失败比较次数之和/表长。

备注：散列表插入结果与 ASL 计算过程详见答案 PDF P6-P7；此处为题面与关键结论登记。

考点　散列表、线性探测、链地址法、ASL

#342　802真题2026

PDF P4　·　真题-2026-综合18



图：a26-p07-0.png

答案

（2）查找成功 ASL=(1×1+2×2+3×4+4×1)/8=21/8=2.625。（1)(3) 树形见答案 PDF 附图。

解析

按 BST 插入规则逐个插入；ASL 按各结点所在层数加权平均；AVL 插入时旋转平衡。

备注：BST 与 AVL 树形图见答案 PDF P7(a26-p07-0.png)。

考点　二叉排序树BST、ASL、平衡二叉树AVL

#343　802真题2026

PDF P5　·　真题-2026-综合23

答案

（2）查找 28 依次比较：25→33→28(mid=(1+11)/2=6→25；28>25向右，区间[7,11] mid=9→33；28<33向左，区间[7,8] mid=7→28，找到)。（1)(3) 判定树与 ASL 见答案。

解析

折半查找判定树按 mid=(low+high)/2 递归构造；比较序列即从根到目标结点的路径。

备注：答案 PDF 注'判定树 word 写不方便，这里不画'，判定树图缺失，按折半过程文字还原比较序列。

考点　折半查找、判定树、ASL

第九章　排序 3 题

#348 408真题

408-2016-43

答案

1）算法的基本设计思想

由题意知，将最小的⌊n/2⌋个元素放在A1中，其余的元素放在A2中，分组结果即可满足题目要求。仿照快速排序的思想，基于枢轴将个整数划分为两个子集。根据划分后枢轴所处的位置i分别处理：

- 若i=⌊n/2⌋，则分组完成，算法结束；
- 若i<⌊n/2⌋，则枢轴及之前的所有元素均属于A1,继续对 i 之后的元素进行划分；
- 若i>⌊n/2⌋，则枢轴及之后的所有元素均属于A2，继续对i之前的元素进行划分；

基于该设计思想实现的算法，无须对全部元素进行全排序，其平均时间复杂度是O(n)，空间复杂度是O(1)。

2）算法实现

实现参考 快速排序的单向递归算法。

```
int partition(int a[], int low, int high) {
int l = low;
int r = high;
int pivot = a[l];
while (l < r) {
while (l < r && a[r] >= pivot) {
r--;
}
a[l] = a[r];
while (l < r && a[l] <= pivot) {
l++;
}
a[r] = a[l];
}
a[l] = pivot;
return l;
}

// 按照第 k 个元素进行分区
void quickSelect(int a[], int low, int high, int k) {
if (low < high) {
int pivotIndex = partition(a, low, high);
if (pivotIndex == k) {
return
} else if (pivotIndex < k) {
quickSelect(a, pivotIndex+1, high, k);
} else {
quickSelect(a, low, pivotIndex-1, k);
}
```

```
}
}

// 空间复杂度：O(1)
// 时间复杂度：O(n)
void solve(int a[], int n) {
quickSelect(a, 0, n-1, n/2);
int S1 = 0;
int S2 = 0;
for (int i = 0; i < n/2; i++) {
S1 += a[i];
}
for (int i = n/2; i < n; i++) {
S2 += a[i];
}
return S2 - S1;
}
```

3）本参考答案给出的算法平均时间复杂度是 O(n)，空间复杂度是O(1) 。

考点　快速排序

#349　408真题

408-2021-42

答案

cmpCountSort 算法基于计数排序的思想，对序列进行排序。cmpCountSort 算法遍历数组 中的元素，count 数组记录比对应待排序数组元素下标大的元素个数，例如，count[1]=3的意思是数组 a 中有 3 个元素比 a[1]大，即 a[1]是第 4 大元素，a[1]的正确位置应是 b[3]。

1）排序结果为 b[6]={-10,10,11,19,25,25}。

2）由代码 for(i=0;i<n-1;i++) 和 for(j=i+1;j<n;j++) 可知，在循环过程中，每 个元素都与它后面的所有元素比较一次（即所有元素都两两比较一次），比较次数之和为(n−1)+(n−2)+⋯+1，故总的比较次数是n(n−1)/2。

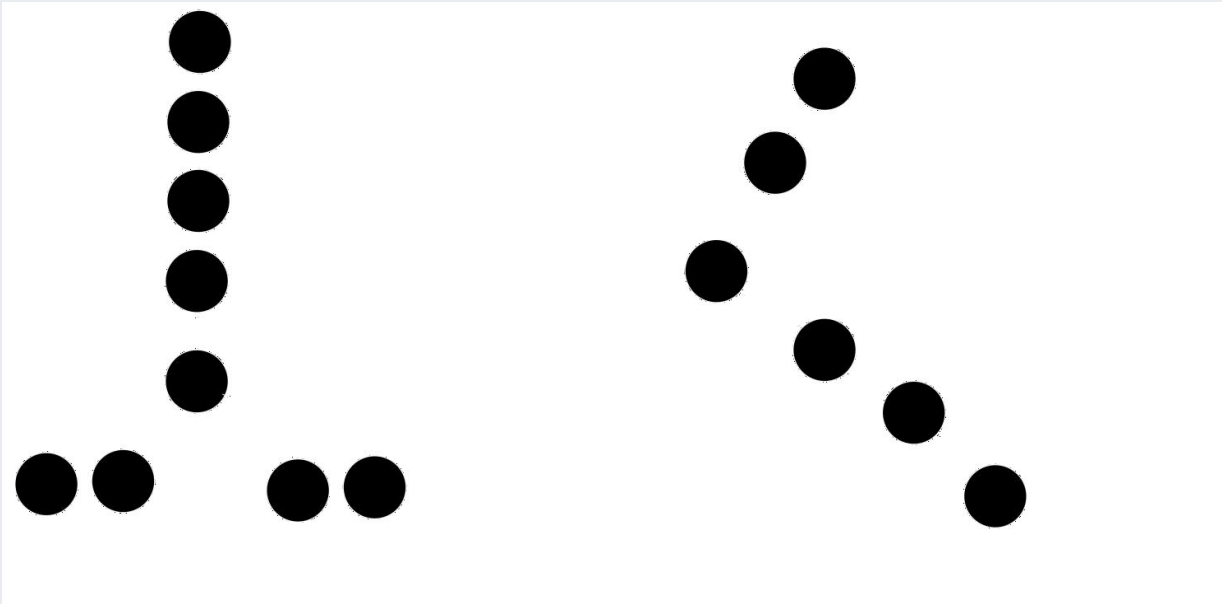
3）不是。需要将程序中的 if 语句修改如下：

```
if(a[i]<=a[j]) count[j]++;
else count[i]++;
```

如果不加等号，两个相等的元素比较时，前面元素的 count 值会加 1，导致原序列中靠前的元素在排序后的序列中处于靠后的位置。

考点　排序算法

#350　802真题2024



图：z2425-p17-0.png

答案

（1）基数排序(补零成三位)： 第1趟(个位)：151, 301, 372, 372, 892, 093, 043, 485, 946, 146, 236, 009 第2趟(十位)：301, 009, 236, 043, 946, 146, 151, 372, 372, 485, 892, 093 第3趟(百位)：009, 043, 093, 146, 151, 236, 301, 372, 372, 485, 892, 946 （2）希尔(增量5)第一趟：43, 301, 372, 9, 146, 93, 892, 485, 236, 372, 151, 946 （3）快排(轴=93)第一趟：43, 9, 93, 372, 146, 151, 301, 485, 236, 372, 946, 892 （4）小根堆：9, 43, 151, 236, 93, 372, 301, 485, 946, 372, 146, 892

解析

（1）LSD 按个/十/百位做稳定收集排序；（2）增量5把下标 i,i+5,i+10 分组组内直接插入；（3）挖坑法以 93 为轴一趟划分；（4）自最后一个非叶结点(n/2−1=5)起向下筛选建小根堆。四问结果均已逐项复核。

【勘误•以修正版为准】 【答案PDF有误】第(1)问第3趟(百位)结果 PDF 写作 {009, 093, 043, 146, …}，其中 093 与 043 顺序颠倒。基数排序为稳定排序且最终须升序，043 < 093，正确结果应为 {009, 043, 093, 146, 151, 236, 301, 372, 372, 485, 892, 946}。第(2)(3)(4)问及第1、2趟均核对无误。排序过程图见答案PDF P17(z2425-p17-0.png)。

考点 基数排序LSD、希尔排序、快速排序、小根堆

算法设计题 9 题

第二章　线性表 2 题

#366 802真题2025

PDF P12 · 真题-2025-算法1

答案

（1）设计思想（快慢指针/双指针一趟扫描）： ① fast、slow 均指向链表首结点； ② 让 fast 先走 k 步，使 fast 与 slow 相隔 k 个结点；若走 k 步过程中 fast 变为 NULL，说明链表长度 < k，返回 0； ③ fast、slow 同步各走一步，直到 fast 到达表尾(NULL)，此时 slow 指向倒数第 k 个结点； ④ 打印 slow->data 并返回 1。

（2）代码：

```
int findKthFromEnd(LinkList head, int k) {
    LinkList fast = head;
    LinkList slow = head;
    // fast 先走 k 步
    for (int i = 0; i < k; i++) {
        if (fast == NULL) {      // 链表长度小于 k
            return 0;
        }
        fast = fast->next;
    }
    // 同步移动直到 fast 到表尾
    while (fast != NULL) {
        fast = fast->next;
        slow = slow->next;
    }
    printf("The %d-th element from the end is: %d\n", k, slow->data);
    return 1;
}
```

（也可用栈或递归实现。）

解析

快慢指针保持固定间距 k，一趟 O(n) 时间、O(1) 空间即可定位倒数第 k 个结点，无需先求链表长度。

备注：答案PDF代码把 head 视为首数据结点(不带头结点)；若链表带头结点，应相应调整起始与长度判定。代码正确。

考点　快慢指针　|　单链表、倒数第k个元素、快慢指针

#367 802真题2026

PDF P5 · 真题-2026-算法24

答案

算法思路：用循环链表模拟——建 n 个结点的循环链表，从第 s 个开始，每数 m-1 步删除当前结点，直到剩最后一个，返回其编号。也可用数组/递推公式(约瑟夫递推 f(n,m)=(f(n-1,m)+m)%n)求解。

解析

经典约瑟夫环。循环链表模拟直观；若只求最后幸存者编号，递推公式 O(n) 更优。

备注：答案 PDF 注：'之前讲冲刺班的时候压到这题了'——与主讲冲刺班押题一致，C3-B 重点讲。完整代码见 C3-B 模板(循环链表模拟)。

考点　循环链表模拟　|　约瑟夫环、循环链表、模拟

第六章　树 4 题

#384 802真题2024

PDF P5 · 真题-2024-算法1

答案

思想：BST 中任一结点的值须落在其允许区间 (min, max) 内——根为 (−∞,+∞)，左子树上界收紧为父值、右子树下界收紧为父值，递归检查全树。

```
bool isBSTUtil(BiTNode* t, int min, int max) {
    if (t == NULL) return true;                 // 空树是 BST
    if (t->data <= min || t->data >= max) return false;
    return isBSTUtil(t->lchild, min, t->data)
        && isBSTUtil(t->rchild, t->data, max);
}
bool isBST(BiTree T) {
    return isBSTUtil(T, INT_MIN, INT_MAX);
}
```

另一等价思路：中序遍历 BST 得严格递增序列，边遍历边比较前驱值即可判定。

解析

范围递归法时间 O(n)、空间 O(树高)。用 INT_MIN/INT_MAX 作初始边界时，若结点值恰等于该极值会有边界误判，工程上可改用 long 或中序遍历法规避。

备注：答案PDF给出范围递归法(isBSTUtil)，逻辑正确。

考点　递归三件套　|　二叉排序树判定、递归、上下界范围

#385 802真题2024

PDF P5 · 真题-2024-算法3

答案

思想：层次遍历，用 last 标记当前层最后一个结点在队列中的位置、levelSize 统计当前层结点数、level 记录层号；当队头到达 last 说明本层遍历完，累加 levelSize×level，然后层号加一、更新 last 为新队尾、levelSize 清零。

```
int calculateProsperity(BiTree T) {
    if (!T) return 0;
    Queue Q; InitQueue(Q);
    int last = 1, level = 1, totalProsperity = 0, levelSize = 0;
    BiTNode* p = T;
    EnQueue(Q, p);
    while (!IsEmpty(Q)) {
        p = DeQueue(Q);
        levelSize++;
        if (p->lchild) EnQueue(Q, p->lchild);
        if (p->rchild) EnQueue(Q, p->rchild);
        if (Q.front == last) {            // 本层结束
```

```
            totalProsperity += levelSize * level;
            level++;
            last = Q.rear;
            levelSize = 0;
        }
    }
    FreeQueue(Q);
    return totalProsperity;
}
```

解析

层序+last 分层模板，时间 O(n)、空间 O(n)。繁荣度 = Σ(每层结点数 × 层号)。

备注：答案PDF代码末行 `FreeQueue(q)` 中 q 应为 Q(笔误)，其余逻辑正确。

考点　层序+last　|　二叉树繁荣度、层次遍历、last分层标记

#386 802真题2025

PDF P12 · 真题-2025-算法2

答案

（1）设计思想（DFS + 路径记录 + 回溯）： ① 从根开始递归遍历，用数组 path 记录当前从根到该结点的路径，pathIndex 记录路径长度； ② 进入结点时把它压入 path；到达叶子结点时，若当前路径长度 currentLen 大于已记录的 maxLen，则更新 maxLen 并输出当前 path； ③ 递归处理左、右子树（currentLen+1）； ④ 回溯：返回前 pathIndex-- 撤销当前结点，保证不同分支路径互不干扰。

（2）代码（答案PDF版，用引用/cout，C++ 风格）：

```
int maxDepth(BiTree root, int& maxLen, int currentLen, int path[], int& pathIndex) {
    if (root == NULL) return 0;
    path[pathIndex] = root->data;   // 当前结点入路径
    pathIndex++;
    if (root->lchild == NULL && root->rchild == NULL) {  // 叶子
        if (currentLen > maxLen) {
            maxLen = currentLen;
            cout << "Longest path: ";
            for (int i = 0; i < pathIndex; i++) cout << path[i] << " ";
            cout << endl;
        }
    }
    int leftLen  = maxDepth(root->lchild,  maxLen, currentLen + 1, path, pathIndex);
    int rightLen = maxDepth(root->rchild, maxLen, currentLen + 1, path, pathIndex);
    pathIndex--;                    // 回溯，撤销当前结点
    return (leftLen > rightLen) ? leftLen : rightLen;
}
// 初值调用示例：maxLen=0, pathIndex=0, maxDepth(root, maxLen, 1, path, pathIndex);
```

解析

先序遍历过程中用 path 保存根到当前结点的路径，遇叶比较并记录最长路径，返回时回溯弹出当前结点。时间 O(n)，空间 O(树高)。属'递归三件套/DFS 回溯'范式。

备注：答案PDF用 C++ 引用(int&)与 cout，题干要求 C；改用 C 可将 maxLen/pathIndex 换成指针传参、cout 换成 printf。逻辑正确。

考点　DFS回溯　|　二叉树、根到叶最长路径、递归、回溯

#387 802真题2026

PDF P5 · 真题-2026-算法25

答案

算法思路：递归划分。先序 A 的第一个元素是根；在中序 B 中找到该根的位置 k，则 B 中 k 左边是左子树的中序、右边是右子树的中序，据左子树结点个数在先序 A 中划分出左右子树的先序；对左右子树递归建树。

解析

先序定根、中序划分左右，是二叉树重建的标准递归。结点值互不相同保证根在中序中位置唯一。

备注：答案 PDF 仅给思路'递归划分'未给完整代码；完整代码见 C3-B 模板(递归建树)，与 2024 判 BST、2025 最长路径同属树的递归范式。

考点　递归建树　|　二叉树重建、先序中序、递归建树

第七章　图 3 题

#393 802真题2024

PDF P5 · 真题-2024-算法2

答案

思想：从 i 出发做深度优先搜索并用 visited 标记，每深入一层路径长度 k 减 1；当到达 j 且剩余长度恰为 0 时说明存在长度 k 的路径，返回 1；一条分支失败则回溯(取消标记)继续尝试其它分支，全部失败返回 0。

```
int visited[MAXSIZE];
int exist_path_len(ALGraph G, int i, int j, int k) {
    if (i == j && k == 0) return 1;
    else if (k > 0) {
        visited[i] = 1;
        for (ArcNode* p = G.vertices[i].firstarc; p; p = p->nextarc) {
            int temp = p->adjvex;
            if (!visited[temp] && exist_path_len(G, temp, j, k - 1)) {
                visited[i] = 0;      // 找到即回溯复位(可选)
                return 1;
            }
        }
        visited[i] = 0;              // 所有邻接分支失败后复位
        return 0;
    }
    return 0;                        // k<0 等情形
}
```

解析

DFS 回溯枚举从 i 出发长度恰为 k 的简单路径。visited 保证路径为简单路径(不重复顶点)；回溯时须在当前顶点所有邻接分支都尝试完毕后再复位 visited[i]。

【勘误•以修正版为准】 【答案PDF有误】PDF 代码把复位语句 `visited[i]=0;` 误放在 for 循环体内部(每尝试一个邻接点后就复位)，与其'思想'所述'递归结束时将当前顶点访问状态重置'矛盾，会导致同一顶点在一条路径中被重复经过、结果错误。正确写法是把 `visited[i]=0;` 移到 for 循环之外、`return 0` 之前(见 answer 修正代码)。

#394　802真题2025

PDF P12　·　真题-2025-算法3

答案

（1）设计思想（分两阶段 DFS）： 阶段一：用 DFS 找从 u 到 k 的一条路径（visited 标记 + path 记录 + 失败回溯）； 阶段二：在阶段一成功的基础上，重置 visited，再用 DFS 找从 k 到 v 的一条路径； 两阶段任一失败则输出 -1；成功则把两段路径拼接输出。

（2）代码（答案PDF版）：

```
bool visited[MaxVertexNum];

// 从 u 出发找到达 k 的路径，记录到 path
bool DFS(ALGraph *G, int u, int v, int k, int *path, int *pathLength) {
    visited[u] = true;
    path[*pathLength] = u;
    (*pathLength)++;
    if (u == k) return true;                  // 到达目标 k
    ArcNode *arc = G->vertices[u].first;
    while (arc != NULL) {
        int w = arc->adjvex;
        if (!visited[w]) {
            if (DFS(G, w, v, k, path, pathLength)) return true;
        }
        arc = arc->next;
    }
    (*pathLength)--;                           // 回溯
    visited[u] = false;
    return false;
}

bool findPath(ALGraph *G, int u, int v, int k) {
    int path[MaxVertexNum];
    int pathLength = 0;
    // 第一步：u -> k
    for (int i = 0; i < MaxVertexNum; i++) visited[i] = false;
    if (!DFS(G, u, v, k, path, &pathLength)) { printf("-1\n"); return false; }
    if (path[pathLength - 1] != k)            { printf("-1\n"); return false; }
    // 第二步：k -> v
    for (int i = 0; i < MaxVertexNum; i++) visited[i] = false;
    if (!DFS(G, k, v, v, path, &pathLength))  { printf("-1\n"); return false; }
    printf("Path: ");
    for (int i = 0; i < pathLength; i++) printf("%d ", path[i]);
    printf("\n");
    return true;
}
```

解析

把'u→v 且经过 k'拆成 u→k 与 k→v 两段简单路径问题，各用一次带回溯的 DFS。visited 防止重复访问成环，path 记录路径，失败时回溯撤销。时间 O(V+E) 每段。

备注：答案PDF代码在第二阶段 DFS(G,k,v,v,...) 仍从 pathLength 处续写，会把顶点 k 再次压入 path，导致输出路径中 k 重复一次；可在第二段起始令 pathLength-- 或跳过重复的 k。此为答案PDF代码的小瑕疵，整体算法思路正确。

#395　802真题2026

PDF P6　·　真题-2026-算法26

```cpp
typedef struct BiTNode {
    char data;
    struct BiTNode *lchild, *rchild;
} BiTNode, *BiTree;

BiTree BuildTree(char *Pre, char *In, int len) {
    if (len <= 0) return NULL;

    BiTNode *root = (BiTNode*)malloc(sizeof(BiTNode));
    root->data = *Pre;

    // 在中序中找根的位置
    int i;
    for (i = 0; i < len; i++) {
        if (In[i] == *Pre) break;
    }

    // 递归构建
    root->lchild = BuildTree(Pre + 1, In, i);
    root->rchild = BuildTree(Pre + 1 + i, In + i + 1, len - i - 1);

    return root;
}
```

图：a26-p10-0.png

```cpp
bool IsTopologicalSort(ALGraph G, int *seq, int n) {
    int indegree[MaxVertexNum] = {0};
    // 1. 计算所有点入度
    for (int i = 0; i < G.vexnum; i++) {
        for (ArcNode *p = G.vertices[i].firstarc; p; p = p->nextarc) {
            indegree[p->adjvex]++;
        }
    }

    // 2. 按seq顺序检查
    for (int i = 0; i < n; i++) {
        int v = seq[i];
        if (indegree[v] != 0) {
            return false; // 当前点入度不为0，不能作为拓扑序的下一个
        }
        // "删除"该点，减少邻接点入度
        for (ArcNode *p = G.vertices[v].firstarc; p; p = p->nextarc) {
            indegree[p->adjvex]--;
        }
    }
    return true;
}
```

图：a26-p11-0.png

答案

算法思路一(前驱检查)：遍历序列中每个顶点 u，检查图中所有指向 u 的顶点 v(入边 v→u)；若存在某前驱 v 在给定序列中出现在 u 的后面，则该序列不是拓扑序列，否则是。 算法思路二(入度表法)：模拟拓扑排序过程，按序列顺序逐个检查当前顶点的入度是否为 0(只统计序列中尚未处理的顶点构成的入度)，若某顶点入度非 0 则不是拓扑序列。

解析

拓扑序列的充要条件：对每条边 (v→u)，v 在序列中都排在 u 前面。两种思路均 O(V+E)(邻接表)。

备注：答案 PDF 给出两种思路(前驱检查、入度表法模拟)并配图(a26-p10-0.png、a26-p11-0.png)。完整代码见 C3-B 模板(入度表法)。

考点　入度表法　|　拓扑排序、拓扑序列验证、入度表